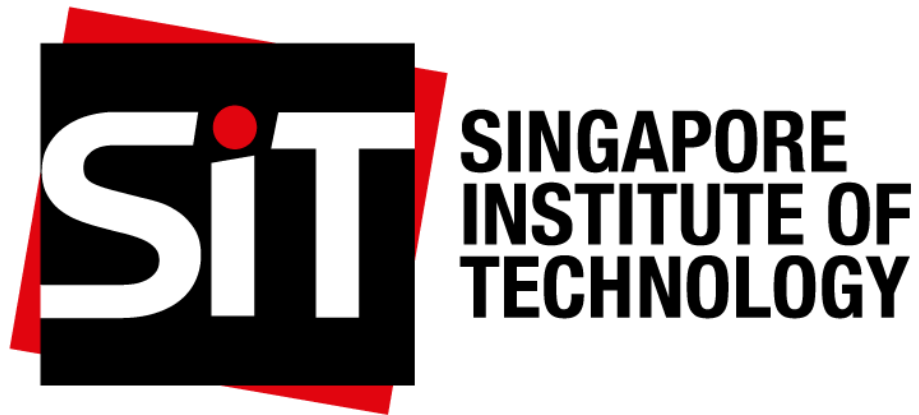




ICT2116 - Secure Software Development

Deliverable One

Group Name: Château Nexus (P2 - Team 31)

Project Title: Château Collective

Name	Student ID
Tason Chua Dong Xuan	2401539
Bryan Toh	2400538
Ong Bang Xi	2400828
Oh Jia Rong	2403275
Cheng Chun Yuan	2402331
Wayne Lee Soo Wei	2400968
Leong Yi Phang	2401341

Table Of Content

1. Overview of Application and Description of Stakeholders	1
1.1. Application Description	1
1.2. Stakeholders and Intended Users	1
1.3. Impact, Creativity, and Usefulness	2
2. Functional & Non-Functional Requirements	3
2.1. Functional Requirements	3
2.2. Non-Functional Requirements	4
3. Security Requirement Analysis	8
3.1 Secure Functional Requirements	8
3.2 Functional Security Requirements	9
3.3 Non-Functional Security Requirements	9
3.4 Secure Development Requirements	10
4. Use Case/Misuse Case Diagrams	10
Summary of Use Case to Misuse Case Mapping	11
UC-01: Account and Profile Management	11
UC-02: Seller Onboarding and Listing Management	12
UC-03: Product Discovery	13
UC-04: Cart, Purchase Commitment, and Simulated Checkout	14
UC-05: Shipment and Order Status Tracking	15
UC-06: Review and Dispute Management	16
UC-07: Admin Monitoring and Enforcement	17
5. Potential risk of application	19
6. Threat Modelling	19
6.1 Threat Modelling Method	19
6.2 Level 0 Data Flow Diagram	19
6.3 Threat Modelling Assumptions	19
6.4 Threat Modelling Tool Output	20
6.5 Validation of Mitigations	20
6.6 Microsoft Threat Modeling Tool Evidence	21
6.7 Example Threat Properties with Mitigation Note	21
7. Attack Surface Analysis	21
7.1. Attack Surface Identification	21
7.2. Attack Surface Categorisation	22
7.3. Administrative Interfaces	22
7.4. Search and Inquiry Functions	22
7.5. Data Entry and CRUD Forms	22
7.6. File Upload Interfaces	23
7.7. Business Workflow Interfaces	23
7.8. Interfaces with Other System Components	23

7.9. Trust Boundaries	24
7.10. Attack Surface Reduction Measures	24
8. System Architecture	25
8.1 Architecture Overview and Design Approach	25
8.2 High-Level Architecture	25
8.3 Logical Architecture	26
8.4 Physical Architecture	26
8.5 Component Architecture	27
9. Security Design	27
9.1. Security Design Principles	27
9.2 Mitigating Threats (STRIDE)	28
9.3. Key Security Design Decisions	28
9.4 Workflow State Design	30
9.5 Class Diagram and Database Model	30
Appendix A	A-1
Appendix A-1 - Secure Functional Requirements	A-1
Appendix A-2 - Functional Security Requirements	A-3
Appendix A-3 - Non-Functional Security Requirements	A-6
Appendix A-4 - Secure Development Requirements	A-9
Appendix B	B-1
Appendix C	C-1
Appendix D	D-1
Appendix E	E-1
Appendix E-1 - System Data Flow	E-1
Appendix E-2 - Threat Modelling	E-3
Appendix F	F-1
Appendix G	G-1
Appendix G-1 - High-Level Architecture	G-1
Appendix G-2 - Logical Architecture	G-4
Appendix G-3 - Physical Architecture	G-8
Appendix G-4 - Component Architecture	G-10
Appendix H	H-1
Appendix H-1 - Security Design Principles	H-1
Appendix H-2 - STRIDE Categories	H-2
Appendix H-3 - Workflow State Design	H-5
Appendix H-4 - Class Diagram	H-7

1. Overview of Application and Description of Stakeholders

1.1. Application Description

Château Collective is a web based second hand luxury goods marketplace designed to connect buyers and sellers of pre-owned luxury items such as handbags, watches, and accessories. The platform enables sellers to list items for sale and buyers to browse, search, and commit to purchases through a structured transaction workflow.

Unlike conventional e-commerce platforms, Château Collective incorporates an in-house authentication workflow. After a buyer commits to a purchase, the seller ships the item directly to Château Collective, where an administrator physically verifies its authenticity before the transaction is allowed to proceed. This ensures that buyers are protected from counterfeit goods and that only verified items reach completion. If an item fails authentication, the transaction is cancelled or disputed accordingly.

Payment within the platform is simulated. No real payment gateway is integrated; instead, the system enforces server-controlled payment state transitions (Pending → Paid → Failed → Refunded) to demonstrate secure transaction handling without real financial risk. This design choice keeps the project scope focused on security correctness over payment infrastructure.

The platform supports four user roles Guest, Buyer, Seller, and Admin, each with clearly defined permissions and access boundaries. Access control is enforced at every layer of the application to prevent privilege escalation, data leakage, and workflow manipulation.

1.2. Stakeholders and Intended Users

The following stakeholders interact with Château Collective, each with distinct responsibilities and access rights:

Stakeholder	Role	Key Permissions
Guest	Unauthenticated visitor	Browse and search public listings, view product details, register an account, log in
Buyer	Authenticated platform user	Manage profile, add items to cart, commit to purchases, track orders, submit reviews for completed purchases, raise dispute requests
Seller	Verified user approved by admin	Create and edit own listings, upload product images, manage shipments after buyer commitment. Cannot

		approve own items or change payment status
Admin	Internal platform operator	Manage users, suspend accounts, monitor listings, update item authentication status after in-house verification, resolve disputes, review audit logs

Each stakeholder group has clearly bounded permissions. Sellers cannot authenticate their own items or access buyer private data beyond what is needed for fulfillment. Admins cannot bypass audit logging for sensitive actions or approve workflow transitions that have not been properly initiated. This separation of duties is a core design principle throughout the platform.

1.3. Impact, Creativity, and Usefulness

1.3.1. Real-World Impact

Counterfeit luxury goods are a significant and growing problem in the second-hand market. Buyers frequently have no reliable way to verify the authenticity of high-value items before completing a purchase. Château Collective addresses this by placing authentication at the centre of the transaction workflow, meaning a buyer cannot complete checkout until an administrator has verified the item. This provides a meaningful layer of consumer protection. By enforcing role-based access control, server-side workflow validation, audit logging, and file upload protection, Château Collective models how a real marketplace can be built with security as a foundational concern rather than an afterthought.

1.3.2. Creativity

The in-house authentication workflow is the creative centrepiece of the application. Rather than a straightforward purchase flow, every second-hand transaction passes through a multi-stage process as follows: buyer commitment → seller shipment → admin verification → transaction completion or cancellation.

This workflow introduces non-trivial security challenges including:

- Enforcing valid state transitions and preventing illegal workflow jumps.
- Ensuring only admins can update authentication status, with full audit trails.
- Preventing sellers from influencing the outcome of their own item's authentication.
- Handling disputed outcomes in a structured and accountable way.

This goes beyond standard CRUD application design and demonstrates a thoughtful approach to business-logic security.

1.3.3. Usefulness

Château Collective is a realistic and well-scoped project that covers a broad range of security concerns in a single, coherent application. Its usefulness as a security demonstration platform stems from:

- Multiple user roles with meaningfully different permissions, enabling comprehensive access control testing.
- A full transaction lifecycle from browsing through to dispute resolution, covering a wide range of functional and security requirements.
- Rich abuse-case opportunities including order tampering, seller fraud, privilege escalation, and admin misuse.
- File upload functionality requiring extension, MIME type, size, and filename validation.
- Audit logging for all sensitive actions, supporting non-repudiation and accountability.
- A simulated payment system that demonstrates secure state management without real financial dependencies.

Together, these characteristics make Château Collective a strong foundation for demonstrating secure web application design across both functional and non-functional dimensions.

2. Functional & Non-Functional Requirements

2.1. Functional Requirements

2.1.1 Authentication & User Management

FR-01 - User Registration: The system shall allow guests to register an account by providing a name, email address, phone number, and password.

FR-02 - User Authentication: The system shall allow registered users (Buyers, Sellers, Admins) to log in and log out of the platform.

FR-03 - Profile Management: The system shall allow authenticated buyers and sellers to manage and update their profiles

FR-04 - Seller Application: The system shall allow regular users to apply to become a verified seller by submitting a seller application.

2.1.2 Catalog & Product Management

FR-05 - Product Browsing: The system shall allow unauthenticated guests and authenticated users to browse public listings and view product detail pages containing product photos, seller information, and specifications (brand, category, condition, price)

FR-06 - Product Search and Filtering: The system shall allow users to search and filter listings by category, brand, condition, and price.

FR-07 - Listing Creation & Editing: The system shall allow authenticated sellers to create and edit their own listings, including description, brand, condition, price, and image uploads.

2.1.3 Order, Cart & Transaction Workflow

FR-08 - Cart Management: The system shall allow buyers to add, view, and remove items from a personalized shopping cart.

FR-09 - Purchase Commitment: The system shall allow buyers to commit to purchase an item.

FR-10 - Shipment Workflow Tracking: The system shall track and allow sellers to update the shipping status of an item (e.g. "awaiting shipment", "shipped") once a buyer commits to the purchase.

FR-11 - Simulated Checkout: The system shall provide a simulated checkout flow that completes the transaction and marks the order status after all required workflow status checks are satisfied.

2.1.4 Admin Platform Operations

FR-12 - Admin Dashboard: The system shall provide an administrative interface to manage platform operations.

FR-13 - User Suspension: The system shall allow administrators to view and suspend suspicious user accounts.

FR-14 - Listing Monitoring: The system shall allow administrators to review suspicious or reported product listings.

FR-15 - Order & Workflow Status Update: The system shall allow administrators to update the order and transaction workflow state based on current compliance.

2.1.5 Review & Dispute Management

FR-16 - Purchase-Based Review System: The system shall allow buyers to submit ratings and textual review only for items they have successfully purchased and completed.

FR-17 - Dispute Resolution Workflow: The system shall allow buyers to raise a dispute request for an order, and allow administrators to review, manage, and resolve the dispute.

2.2. Non-Functional Requirements

2.2.1. Operational Requirements

NFR-01 - System Availability

Description:

The system shall remain available for guests, buyers, sellers, and administrators during normal operating periods.

Priority: High

Measurement / Metric:

The system shall achieve at least 99% uptime during the project testing or demonstration period.

Verification Method:

Availability testing, uptime monitoring, and repeated access testing.

NFR-02 - Data Integrity

Description:

The system shall maintain accurate and consistent records for users, seller applications, product listings, carts, orders, reviews, and disputes.

Priority: High

Measurement / Metric:

There shall be no unexplained mismatch between database records and displayed information during testing.

Verification Method:

Database validation, integration testing, and data consistency checks.

NFR-03 - Backup and Recovery

Description:

The system shall support backup and recovery of important user, product, order, review, dispute, and admin operation data.

Priority: Medium

Measurement / Metric:

A database backup shall be restorable in the test environment without loss of critical records.

Verification Method:

Backup and restore testing.

NFR-04 - Error Recovery

Description:

The system shall handle failed actions gracefully, including failed registration, listing creation, image upload, cart update, checkout, shipment update, review submission, and dispute submission.

Priority: Medium

Measurement / Metric:

Failed actions shall display a controlled error message and shall not corrupt existing system data.

Verification Method:

Negative testing and fault handling tests.

2.2.2. Performance Requirements

NFR-05 - Page Response Time

Description:

The system shall load common pages within an acceptable response time under normal usage.

Priority: High

Measurement / Metric:

Product browsing, product details, cart, checkout, profile, and admin pages shall load within 3 seconds under normal test load.

Verification Method:

Performance testing and browser response-time measurement.

NFR-06 - Search and Filtering Performance

Description:

The system shall return product search and filtering results within an acceptable time.

Priority: High

Measurement / Metric:

Search and filtering results shall be returned within 2 seconds for the normal project dataset size.

Verification Method:

Search performance testing.

NFR-07 - Image Upload Performance

Description:

The system shall process product image uploads within an acceptable time when the uploaded file meets the allowed file size and format restrictions.

Priority: Medium

Measurement / Metric:

Valid product image uploads shall complete within 5 seconds under normal network and server conditions.

Verification Method:

File upload performance testing.

NFR-08 - Concurrent User Capacity

Description:

The system shall support multiple guests, buyers, sellers, and administrators using the platform at the same time.

Priority: Medium

Measurement / Metric:

The system shall support at least 50 concurrent users in the test environment without major errors or unacceptable slowdown.

Verification Method:

Load testing.

NFR-09 - Transaction Processing Performance

Description:

The system shall process cart updates, purchase commitment, shipment status updates, and simulated checkout actions without noticeable delay.

Priority: High

Measurement / Metric:

Cart update, purchase commitment, shipment update, and checkout actions shall complete within 3 seconds under normal test load.

Verification Method:

End-to-end performance testing.

NFR-10 - Scalability of Product Listings

Description:

The system shall continue to perform acceptably as the number of product listings increases.

Priority: Medium

Measurement / Metric:

Product browsing, searching, and filtering shall remain within the required response-time target when the database contains at least 1,000 product listings.

Verification Method:

Data-volume testing and load testing.

2.2.3. System Maintainability Requirements

NFR-11 - Observability

Description:

The system shall provide sufficient logs to support debugging and monitoring of major workflows.

Priority: Medium

Measurement / Metric:

The system shall record meaningful logs for failed login attempts, listing creation, purchase commitment, shipment updates, checkout, dispute creation, and admin actions.

Verification Method:

Log inspection during system testing.

NFR-12 - Browser Compatibility

Description:

The web application shall work correctly on common modern browsers.

Priority: Medium

Measurement / Metric:

The application shall be usable on the latest versions and the last two major version of Google Chrome, Microsoft Edge, and Mozilla Firefox.

Verification Method:

Cross-browser testing.

2.2.4. Privacy and Compliance Requirements

NFR-13 - Personal Data Minimisation

Description:

The system shall only collect personal data required for account registration, profile management, seller application, order processing, review handling, and dispute management.

Priority: High

Measurement / Metric:

Registration, profile, seller application, and dispute forms shall not request unnecessary personal information.

Verification Method:

Requirements review and form review.

NFR-14 - Privacy of User Records

Description:

The system shall prevent users from viewing private records belonging to other users, including profile details, cart contents, order history, reviews awaiting eligibility, and dispute details.

Priority: High

Measurement / Metric:

Unauthorised attempts to access another user's records shall be blocked during testing.

Verification Method:

Access control testing and negative testing.

NFR-15 - Retention of Business Records

Description:

The system shall retain important order, shipment, review, dispute, and admin operation records for accountability during the project lifecycle.

Priority: Medium

Measurement / Metric:

Completed orders, reviews, disputes, and admin actions shall remain retrievable after workflow completion.

Verification Method:

Database inspection and record retrieval testing.

NFR-16 - Auditability of Admin Actions

Description:

Important administrator actions shall be traceable for accountability and investigation purposes.

Priority: High

Measurement / Metric:

User suspension, listing monitoring, order workflow updates, and dispute resolution actions shall be recorded with administrator identity, timestamp, action type, and target record.

Verification Method:

Audit log review and admin workflow testing.

3. Security Requirement Analysis

See in [Appendix A-1](#) for full Secure Functional Requirements.

See in [Appendix A-2](#) for full Functional Security Requirements.

See in [Appendix A-3](#) for full Non-Functional Security Requirements.

See in [Appendix A-4](#) for full Secure Development Requirements.

3.1 Secure Functional Requirements

This section focuses on the security-related functions that the system must provide during normal use. It covers secure registration and login, profile and seller application management, safe product browsing and searching, controlled listing creation and editing, protected cart and purchase workflows, secure shipment updates, admin-only operations, and review and dispute handling based on valid purchases and ownership. Taken together, these requirements ensure

that each user role can only access the functions and records that are appropriate to that role, while preventing unauthorized viewing, editing, and workflow manipulation.

One example is **SFR-03 Secure Profile Management**. This requirement is important because profile information contains personal data such as contact details and account information. The system must make sure that an authenticated user can edit only their own profile and cannot view or change another user's details. This helps protect privacy and prevents users from interfering with accounts that do not belong to them.

Another example is **SFR-16 Secure Purchase-Based Review System**. This requirement is identified because reviews should reflect real purchasing experience rather than fake feedback. If any user could submit a review without buying the item, the review system could be misused to damage seller reputation or inflate product ratings. By limiting reviews to completed purchases, the system ensures that feedback is authentic and linked to real transactions.

3.2 Functional Security Requirements

This section defines the main security controls built into the system's functionality. It includes administrator two-factor authentication, password security rules, secure password storage, session management, brute-force protection, and account abuse prevention. It also covers role-based access control, audit logging, protection of private records, data minimisation, secure handling of simulated payment data, encryption in transit and at rest, security headers, CSRF protection, server-side business rule enforcement, traffic filtering, and backup and recovery support. These requirements ensure the system is protected against unauthorised access, misuse, data exposure, and loss of integrity.

A clear example is **FSR-11 Audit Logging**. This requirement is needed because important actions must be traceable after they happen. For example, if a seller edits a listing or an admin suspends an account, the system should record that action so it can later be reviewed if there is a dispute or security incident. Audit logs support accountability because they show what happened, when it happened, and who performed the action.

Another example is **FSR-05 Brute Force Protection**. This requirement is included because attackers often try many password combinations to guess an account password. Without protection, repeated login attempts could place user accounts at risk. By slowing down or temporarily restricting repeated failed attempts, the system reduces the chance of unauthorised access through password guessing.

3.3 Non-Functional Security Requirements

This section describes the security qualities the system must maintain during operation and testing. It covers confidentiality, integrity, availability, performance, auditability, observability, and error handling. These requirements focus on how securely and reliably the system behaves, rather than on the exact features it provides. They ensure that private data stays protected,

records remain accurate, the system remains available, pages respond within acceptable time limits, logs remain useful, alerts are generated for suspicious activity, and error messages do not reveal sensitive technical details.

One example is **NFSR-21 Secure Error Messages**. This requirement is important because error messages can accidentally expose technical details such as database errors, stack traces, or server file paths. Such information can help an attacker understand how the system is built and where weaknesses may exist. Controlled error messages prevent this type of information leakage while still allowing developers to keep detailed diagnostics in secure server logs.

Another example is **NFSR-09 System Availability**. This requirement is identified because the system must remain usable during the project testing and demonstration period. If the application goes down too often, users would not be able to browse products, place orders, or complete other workflow tasks. Requiring a minimum uptime level helps show that the system is dependable enough for normal use.

3.4 Secure Development Requirements

This section covers the secure practices that developers must follow when building, testing, and deploying the system. It includes input validation, output encoding, SQL injection prevention, secure file upload handling, password and session implementation, access control testing, negative testing, audit log testing, backup testing, secrets management, secure deployment configuration, server hardening, traffic filtering, secure code review, dependency review, browser compatibility testing, and controlled logging and error handling. These requirements ensure that security is considered throughout the development process instead of being added only after the system is completed.

A good example is **SDR-04 Secure File Upload Handling**. This requirement is needed because file upload features are often abused if they are not carefully restricted. If file type, file size, extension, MIME type, and storage location are not controlled, an attacker could upload unsafe files or attempt to place malicious content on the server. Secure upload handling reduces this risk by making sure only valid image files are accepted and stored safely.

Another example is **SDR-11 Secrets Management**. This requirement is identified because sensitive information such as database passwords, API keys, and private tokens must not be exposed in source code or public repositories. If secrets are hardcoded or stored carelessly, they can be leaked and used to gain unauthorised access to the system. Proper secrets management protects these credentials during development, deployment, and maintenance.

4. Use Case/Misuse Case Diagrams

This section identifies the main use cases and misuse cases for Château Collective. The normal use cases describe the intended business functions of the marketplace, while the misuse cases describe how attackers, rogue users, or compromised accounts may abuse these functions.

Summary of Use Case to Misuse Case Mapping

With reference to [Table B1](#) for table of use case mapping to main misuse case and main security focus.

UC-01: Account and Profile Management

With reference to [Figure C1](#) for use case diagram.

Mapped requirements: FR-01, FR-02, FR-03

Goal: Allow guests to register an account, allow registered users to log in and log out, and allow authenticated buyers and sellers to manage their own profile information.

Primary actors: Guest, Buyer, Seller, Admin

Preconditions:

- Guest is not logged in before registration.
- Buyer, Seller, or Admin has an existing account before login.
- Buyer or Seller is authenticated before profile management.

Main success flow:

1. Guest opens the registration page.
2. Guest enters name, email address, phone number, and password.
3. System validates the registration details.
4. System creates the user account.
5. Registered user logs in using valid credentials.
6. System authenticates the user and creates a valid session.
7. Buyer or Seller updates their own profile information.
8. System validates and saves the profile changes.

Postconditions:

- A new account is created, or an existing user session is active.
- Profile information is updated successfully.
- The user cannot access another user's private profile information.

Misuse Case: Account Takeover, Fake Registration and Profile IDOR

Threat actor: External attacker, bot, compromised user

Misuse goal: Create fake accounts, perform credential stuffing or brute-force attacks, hijack sessions, or modify another user's profile by changing user identifiers in requests.

Threatened use cases:

- Register Account
- Log In / Log Out
- Manage Profile

Potential impact:

- Unauthorised account access
- Fake user accounts
- Exposure or modification of personal data
- Loss of trust and accountability
- Further abuse of marketplace functions using compromised accounts

Mitigations:

- Enforce strong password rules and secure password hashing.

- Apply rate limiting and progressive throttling to registration and login attempts.
- Use secure session management with logout and inactivity timeout.
- Enforce object-level access control so users can only view or update their own profile.
- Log failed login attempts, suspicious account activity, and profile changes.
- Apply CSRF protection to profile update actions.

UC-02: Seller Onboarding and Listing Management

With reference to [Figure C2](#) for use case diagram.

Mapped requirements: FR-04, FR-07

Goal: Allow a registered user to apply to become a seller, allow administrators to review the application, and allow verified sellers to create or edit their own product listings.

Primary actors: Registered User, Seller, Admin

Preconditions:

- Registered user is logged in before applying to become a seller.
- Admin is authenticated before reviewing seller applications.
- Seller is verified before creating or editing listings.
- Seller can only manage listings owned by their own seller account.

Main success flow:

1. Registered user submits a seller application.
2. System stores the seller application for review.
3. Admin reviews the seller application.
4. Admin approves or rejects the seller application.
5. If approved, the user becomes a verified seller.
6. Verified seller creates a listing with product details and images.
7. Seller edits their own listing if required.
8. System validates and saves the listing changes.
9. Listing remains subject to admin review or approval rules before becoming publicly visible.

Postconditions:

- Seller status is updated after admin review.
- Listing is created or edited by the authorised seller.
- Second-hand listings remain controlled by the required approval workflow.

Misuse Case: Fraudulent Seller, Counterfeit Listing and Malicious Upload Abuse

Threat actor: External attacker, rogue seller, compromised seller account

Misuse goal: Submit fraudulent seller applications, bypass the seller approval process, publish counterfeit or misleading listings, upload malicious files as product images, or edit another seller's listing.

Threatened use cases:

- Apply to Become Seller
- Review Seller Application
- Create Listing
- Edit Own Listing

Potential impact:

- Counterfeit product listings
- Seller fraud
- Malicious file upload
- Unauthorised listing modification
- Exposure of buyers to deceptive or harmful listings
- Loss of platform trust

Mitigations:

- Require admin review before seller approval.
- Enforce seller role checks before listing creation.
- Enforce listing ownership checks before editing.
- Keep second-hand listings hidden until admin approval.
- Validate uploaded files by file type, extension, size, and content.
- Store uploaded files outside public executable directories.
- Log seller application decisions, listing creation, listing edits, and rejected uploads.
- Apply moderation controls for suspicious listings

UC-03: Product Discovery

With reference to [Figure C3](#) for use case diagram.

Mapped requirements: FR-05, FR-06

Goal: Allow guests and registered users to browse public listings, view product details, search listings, and filter listings by category, brand, condition, and price.

Primary actors: Guest, Buyer, Seller, Admin

Preconditions:

- Product listings exist in the system.
- Only approved public listings are displayed to normal users.
- Internal listing flags, admin notes, and private seller information are not exposed publicly.

Main success flow:

1. User opens the catalogue or search page.
2. System displays approved public listings.
3. User views product details.
4. User enters search terms or applies filters.
5. System validates search and filter input.
6. System returns matching public listings.

Postconditions:

- Relevant public products are displayed.
- Private or internal listing data remains hidden.

Misuse Case: Search Injection, Listing Enumeration and Scraping Abuse

Threat actor: External attacker, bot, scraper

Misuse goal: Inject malicious search input, enumerate hidden or unapproved listings, scrape product or seller information at scale, or flood search and product detail endpoints.

Threatened use cases:

- Browse Products
- View Product Details
- Search / Filter Listings

Potential impact:

- Data harvesting
- Leakage of unpublished listings or internal listing metadata
- Slow search performance
- Service degradation
- Possible injection attacks

Mitigations:

- Validate and sanitise search and filter input.
- Use parameterised queries for database access.
- Ensure only approved public listings are returned.
- Hide internal listing flags, admin review notes, and private seller information.
- Apply rate limiting and request throttling to search and product detail endpoints.
- Detect and restrict abnormal automated traffic.
- Log suspicious search activity.

UC-04: Cart, Purchase Commitment, and Simulated Checkout

With reference to [Figure C4](#) for use case diagram.

Mapped requirements: FR-08, FR-09, FR-11

Goal: Allow buyers to manage their cart, commit to purchase an item, and complete the simulated checkout only when the required workflow rules are satisfied.

Primary actor: Buyer

Preconditions:

- Buyer is logged in.
- Product is approved, visible, and available.
- Buyer has permission to manage only their own cart.
- Checkout can proceed only after the required order workflow conditions are satisfied.

Main success flow:

1. Buyer adds an available item to their cart.
2. Buyer views or removes items from their own cart.
3. Buyer commits to purchase an item.
4. System verifies item availability, cart ownership, and order ownership.
5. System creates or updates the purchase commitment.
6. System checks required workflow states before simulated checkout.
7. System completes simulated checkout only when all required workflow checks are satisfied.
8. System marks the order with the correct final simulated status.

Postconditions:

- Cart changes are saved for the correct buyer.
- Purchase commitment is created only for an available item.
- Simulated checkout does not bypass required workflow rules.
- Order state remains consistent.

Misuse Case: Price Tampering, Order Ownership Bypass and Workflow Skipping

Threat actor: External attacker, dishonest buyer, compromised buyer account

Misuse goal: Modify cart values, tamper with item price or availability, access another buyer's cart or order, reserve items unfairly, bypass purchase commitment checks, or force checkout without satisfying the required workflow status.

Threatened use cases:

- Manage Cart
- Commit to Purchase
- Simulated Checkout

Potential impact:

- Invalid orders
- Price manipulation
- Unauthorised access to another buyer's cart or order
- Inconsistent transaction state
- Unfair reservation of high-value items
- Bypass of business workflow controls

Mitigations:

- Enforce server-side validation for price, item availability, cart ownership, order ownership, and workflow status.
- Never rely on client-side price, item status, or hidden form values.
- Apply transaction locking or atomic updates to prevent duplicate reservation.
- Enforce approved state transitions before checkout.
- Apply CSRF protection to cart update, purchase commitment, and checkout actions.
- Log cart updates, purchase commitments, failed checkout attempts, and suspicious workflow bypass attempts.

UC-05: Shipment and Order Status Tracking

With reference to [Figure C5](#) for use case diagram.

Mapped requirements: FR-10, FR-15

Goal: Allow sellers to update shipment status for their own orders, allow buyers to view order status, and allow administrators to update workflow status when required.

Primary actors: Seller, Admin, Buyer

Preconditions:

- A purchase commitment or order exists.
- Seller is authenticated and owns the listing related to the order.
- Buyer is authenticated and owns the order.
- Admin is authenticated before performing workflow updates.

Main success flow:

1. Seller opens an order related to their own listing.
2. Seller updates shipment status, such as awaiting shipment or shipped.
3. System validates that the seller is authorised to update the shipment.

4. Admin reviews compliance or authentication workflow status if required.
5. Admin updates workflow status when appropriate.
6. Buyer views the updated order or shipment status.

Postconditions:

- Shipment and order status reflect the latest authorised workflow state.
- Buyers, sellers, and admins only access records they are authorised to view or update.
- Important workflow changes are traceable.

Misuse Case: False Shipment Update and Unauthorised Workflow State Change

Threat actor: Rogue seller, malicious admin, compromised seller account, compromised admin account

Misuse goal: Mark an item as shipped when it has not been shipped, update shipment records for unrelated orders, skip required workflow states, or improperly override order status.

Threatened use cases:

- Update Shipment Status
- View Order Status
- Update Workflow Status

Potential impact:

- Fraudulent shipment records
- Incorrect order status
- Customer disputes
- Bypass of authentication or compliance workflow
- Loss of accountability

Mitigations:

- Enforce seller ownership checks before shipment updates.
- Enforce buyer ownership checks before viewing order status.
- Restrict workflow status updates to authorised administrators.
- Validate allowed state transitions.
- Require reason or evidence fields for sensitive workflow changes where appropriate.
- Log shipment updates, workflow changes, administrator identity, timestamp, action type, and target record.
- Protect audit logs from unauthorized modification or deletion.

UC-06: Review and Dispute Management

With reference to [Figure C6](#) for use case diagram.

Mapped requirements: FR-16, FR-17

Goal: Allow buyers to submit reviews only for completed purchases and raise disputes only for their own orders, while allowing administrators to review and resolve disputes.

Primary actors: Buyer, Admin

Preconditions:

- Buyer is authenticated.
- Buyer has a completed purchase before submitting a review.

- Buyer has a valid related order before raising a dispute.
- Admin is authenticated before dispute handling.

Main success flow:

1. Buyer opens the review form for a completed order.
2. System verifies purchase and review eligibility.
3. Buyer submits rating and textual review.
4. Buyer raises a dispute for their own order if required.
5. System verifies order ownership before accepting the dispute.
6. Admin reviews the dispute.
7. Admin records the dispute outcome.
8. System stores the review or dispute record for traceability.

Postconditions:

- Review is stored only if the buyer is eligible.
- Dispute is stored only if it relates to the buyer's own order.
- Admin dispute resolution action is traceable.

Misuse Case: Fake Review, Invalid Dispute and Dispute Abuse

Threat actor: Attacker, dishonest buyer, compromised buyer account

Misuse goal: Post reviews without a completed purchase, submit multiple or automated reviews, raise disputes for unrelated orders, submit frivolous disputes to pressure sellers, or tamper with dispute details.

Threatened use cases:

- Submit Verified Review
- Raise Dispute
- Resolve Dispute

Potential impact:

- Reputation manipulation
- Unfair seller damage
- Operational burden
- Incorrect dispute records
- Loss of marketplace trust

Mitigations:

- Verify purchase completion before allowing review submission.
- Enforce one eligible review per completed purchase where appropriate.
- Verify order ownership before allowing dispute submission.
- Require dispute reason and supporting details.
- Monitor repeated or abnormal dispute submissions.
- Restrict dispute resolution to authorised administrators.
- Log review submission, dispute creation, and dispute resolution actions.

UC-07: Admin Monitoring and Enforcement

With reference to [Figure C7](#) for use case diagram.

Mapped requirements: FR-12, FR-13, FR-14, FR-15, FR-17

Goal: Allow administrators to manage platform operations, review suspicious users or listings, suspend users, update workflow or dispute records, and ensure enforcement actions are traceable.

Primary actor: Admin

Preconditions:

- Admin is authenticated.
- Admin has the required administrative role.
- Sensitive administrative actions are logged.

Main success flow:

1. Admin is authenticated.
2. Admin has the required administrative role.
3. Sensitive administrative actions are logged.

Postconditions:

- Platform enforcement action is completed.
- Suspicious users, listings, workflow states, or disputes are updated.
- Administrative actions are traceable through protected audit logs.

Misuse Case: Admin Account Takeover, Abusive Admin Action and Audit Log Tampering

Threat actor: External attacker, compromised admin, malicious admin

Misuse goal: Access the admin dashboard without authorisation, take over an admin account, suspend users unfairly, approve or hide suspicious listings improperly, alter workflow or dispute records, or modify/delete audit logs to hide abuse.

Threatened use cases:

- Access Admin Dashboard
- Review Suspicious Users / Listings
- Suspend User Account
- Update Workflow / Dispute Records
- Review Audit Logs
- Record Audit Trail

Potential impact:

- Full or partial platform compromise
- Abuse of privileged functionality
- Unfair user suspension
- Fraudulent listing or dispute decisions
- Loss of accountability
- Audit trail destruction
- Damage to marketplace trust

Mitigations:

- Require strong administrator authentication, preferably multi-factor authentication.
- Enforce role-based access control for administrative functions.
- Apply least privilege to admin permissions.
- Require confirmation or approval for highly sensitive actions where appropriate.
- Record administrator identity, timestamp, action type, and target record for all important admin actions.

- Protect audit logs from unauthorized modification or deletion.
- Monitor unusual admin activity.
- Apply CSRF protection to all admin state-changing actions.

5. Potential risk of application

Risks have been identified in [Table D1](#) by risk name, description and grading the levels of likelihood, impact and criticality with Low, Medium and High.

6. Threat Modelling

6.1 Threat Modelling Method

The focus of this threat model is on access control, file upload security, session management, order tampering, seller fraud, admin abuse, audit logging, workflow state validation, and simulated authentication status protection.

The system does not claim to truly authenticate luxury goods. Instead, Château Collective supports an admin-managed in-house authentication workflow after buyer commitment. Actual physical verification is simulated for this project.

The finalised misuse cases from Section 4 are used as the main reference for reviewing the generated STRIDE threats. These include account takeover and profile IDOR, fraudulent seller and malicious upload abuse, search injection and listing enumeration, price tampering and workflow skipping, false shipment or workflow updates, fake review and dispute abuse, and admin account takeover or audit log tampering.

6.2 Level 0 Data Flow Diagram

The Level 0 DFD represents Château Collective as one main web application process. External users interact with the web application through browser requests and responses, while the web application communicates with the database, uploaded image storage, and audit log storage.

The diagram is kept at system-context level. Detailed misuse scenarios such as object ID tampering, malicious upload, checkout workflow bypass, invalid review/dispute submission, admin abuse, and audit log tampering are analysed in the STRIDE threat summary instead of being drawn as separate Level 0 processes.

The Level 0 DFD is shown in [Figure E-1.1](#), and the DFD element summary is provided in [Table E-1.1](#).

6.3 Threat Modelling Assumptions

1. Château Collective is a browser-based Flask web application accessed by guests, buyers, sellers, and administrators.
2. The system uses SQLite as the application database and stores uploaded product images separately from normal application data.

3. The project uses simulated checkout and does not process real payment card numbers, CVV values, or banking credentials.
4. The system supports a simulated admin-managed authentication workflow for second-hand luxury goods.
5. Users may attempt to tamper with requests, object IDs, prices, order status, role values, file uploads, and workflow states.
6. Administrators are privileged users, but admin abuse or admin account compromise is still considered a threat.
7. The SQLite database, uploaded image storage, and audit log store are treated as protected backend storage components.
8. Security controls such as role-based access control, ownership checks, file upload validation, audit logging, and server-side workflow validation are expected to be implemented and tested.
9. Automated bots may attempt fake registration, credential stuffing, scraping, repeated search requests, repeated uploads, and repeated dispute submissions.
10. Compromised buyer, seller, and administrator accounts are considered valid threat sources.
11. Object ID manipulation is considered a threat across profile, listing, cart, order, shipment, review, and dispute records.
12. Audit logs are treated as security-sensitive records because they support accountability and investigation.

6.4 Threat Modelling Tool Output

Microsoft Threat Modeling Tool generated STRIDE-based threats from the Level 0 DFD. The repeated tool-generated threats were reviewed and grouped by project relevance instead of being listed individually in the main report.

The main threats selected for analysis relate to account spoofing, request tampering, malicious file upload, unauthorised record access, admin privilege abuse, audit log integrity, workflow tampering, and excessive request abuse.

Azure-specific threats that were not applicable to the Flask and SQLite implementation were marked as not applicable. Relevant threats were updated with mitigation notes and prioritised based on confidentiality, integrity, availability, and accountability.

Screenshots of the generated STRIDE threat list and an example Threat Properties panel with mitigation notes are provided in [Appendix E-2](#). Additional review was performed against the finalised misuse cases in Section 4 to ensure that project-specific threats such as profile IDOR, listing approval bypass, malicious upload, search enumeration, checkout workflow skipping, fake review or dispute abuse, and audit log tampering were considered even where the tool-generated threats were more generic.

6.5 Validation of Mitigations

The proposed mitigations will be validated through security testing, negative testing, SAST, and DAST. Access control testing will include direct URL manipulation, object ID manipulation, and attempts to access buyer, seller, or admin functions without the correct role.

File upload testing will check invalid extensions, incorrect MIME types, oversized files, and files disguised as images. Request tampering tests will check whether users can modify prices, order status, role values, simulated authentication status, or workflow states through browser requests.

Audit log review will verify that key actions such as login attempts, listing updates, shipment updates, workflow changes, dispute handling, and admin actions are recorded with sufficient traceability. Additional misuse-case testing will cover fake reviews, invalid disputes, listing approval bypass, search injection, excessive requests, and audit log tampering.

6.6 Microsoft Threat Modeling Tool Evidence

With reference to [Figure E-2.1](#) for evidence of the generated STRIDE Threat List, the screenshot shows the STRIDE-based threats generated from the Level 0 DFD. The generated threats were reviewed and grouped into the main threat categories used in Section 6.

6.7 Example Threat Properties with Mitigation Note

With reference to [Figure E-2.2](#), the screenshot shows an example of a generated STRIDE threat opened in Microsoft Threat Modeling Tool. The selected threat is related to a Guest Request, where requests from third-party domains may result in unauthorised actions or access to data.

Example threat selected: Guest Request - Denial of Service / Cross-Domain Request Abuse

Status: Mitigated

Severity: High

Mitigation note:

Mitigated by enforcing CSRF protection on authenticated state-changing requests, restricting CORS to trusted origins only, validating Origin/Referer headers where applicable, using secure SameSite session cookies, and applying server-side access control checks.

7. Attack Surface Analysis

For Château Collective, the attack surface includes user interfaces, authentication mechanisms, file uploads, search functions, transaction workflows, administrative functions, databases, and storage systems. Since the platform supports multiple user roles and handles high-value luxury goods transactions, reducing the attack surface is essential to prevent fraud, unauthorised access, workflow manipulation, and data exposure.

7.1. Attack Surface Identification

The first step is to identify all entry and exit points through which data enters or leaves the system. This has been done and is reflected on [Table F1](#).

7.2. Attack Surface Categorisation

To simplify analysis, attack surfaces are grouped according to their function within the system.

7.2.1 Authentication Entry Points

Authentication entry points are responsible for verifying user identities and controlling access to protected resources.

Entry Points	User Registration, User Login, User Logout, Session Cookies, Administrator Authentication
Potential Threats	Brute-force attacks, Credential stuffing, Account takeover, Session hijacking, Session fixation, User enumeration
Security Controls	Password hashing with salt, Strong password policy, Login rate limiting, Account lockout mechanisms, Session expiry after inactivity, Secure cookie attributes, Two-factor authentication for administrators

7.3. Administrative Interfaces

Administrative interfaces provide privileged access to platform management functions and therefore represent a critical attack surface.

Entry Points	Admin Dashboard, User Suspension Functions, Listing Moderation Functions, Workflow Update Functions, Audit Log Viewer
Potential Threats	Privilege escalation, unauthorised admin access, insider abuse, administrative account compromise, audit log tampering
Security Controls	RBAC, administrator authentication, least privilege, separation of duties, immutable audit logging, activity monitoring

7.4. Search and Inquiry Functions

Search functions are publicly accessible and therefore form part of the external attack surface.

Entry Points	Product Search Bar, Product Filters, Product Detail Pages
Potential Threats	SQL injection, search flooding, data scraping, stored XSS
Security Controls	Parameterised queries, input validation, output encoding, request throttling, pagination controls, rate limiting

7.5. Data Entry and CRUD Forms

The system contains multiple forms that allow users to create, update, or submit information.

Entry Points	Registration Form, Profile Update Form, Seller Application Form, Product Listing Form, Product Editing Form, Review Submission Form, Dispute Submission Form
Potential Threats	SQL injection, XSS, parameter tampering, mass assignment attacks, fraudulent submissions
Security Controls	Server-side validation, input sanitisation, output encoding, ownership validation, anti-automation controls

7.6. File Upload Interfaces

File upload functionality allows sellers to upload product images and represents one of the highest-risk attack surfaces.

Entry Points	Product Image Upload
Potential Threats	Malicious file uploads, remote code execution, path traversal, oversized uploads, file overwrite attacks
Security Controls	File type validation, MIME verification, file signature validation, file size restrictions, server-generated filenames, secure storage, safe file serving

7.7. Business Workflow Interfaces

Business workflows control how transactions move through the marketplace.

Entry Points	Purchase Commitment, Shipment Updates, Authentication Status Updates, Checkout Processing, Order Status Updates
Potential Threats	Workflow bypass, price tampering, false shipment declarations, state manipulation, checkout bypass
Security Controls	Server-side workflow validation, transaction integrity controls, ownership verification, server-side price calculation, audit logging, state transition restrictions
Workflow States	Available, Committed, Awaiting Shipment, Shipped, Under Authentication, Authenticated, Rejected, Sold

Only authorised users should be permitted to perform valid state transitions.

7.8. Interfaces with Other System Components

The application communicates with multiple databases and storage components.

Entry Points	User Database, Product & Listing Database, Cart & Order Database, Review & Dispute Database, Audit Log Database, Uploaded Image Storage
Potential Threats	Unauthorised database access, information disclosure, data tampering, audit log manipulation
Security Controls	Access control enforcement, database authentication, data validation, audit logging, least privilege permissions

7.9. Trust Boundaries

The Château Collective system contains several trust boundaries where information moves between trusted and untrusted environments.

External User Boundary

Separates Guests, Buyers, Sellers, and Admins from the web application. All requests crossing this boundary must be validated, authenticated, and authorised.

Application Boundary

Separates user-facing application components from backend business logic and workflow processing.

Data Storage Boundary

Separates application services from databases and uploaded image storage systems.

Administrative Boundary

Separates privileged administrative functions from standard user functionality.

7.10. Attack Surface Reduction Measures

Based on the identified attack surfaces, Château Collective applies the following security principles:

- Minimise attack surface area through role restrictions.
- Establish secure defaults for authentication and session management.
- Apply least privilege for buyers, sellers, and administrators.
- Implement defence in depth using multiple security controls.
- Validate all user input before processing.
- Enforce secure file upload handling.
- Protect workflow integrity through server-side validation.
- Maintain immutable audit logs for accountability.

8. System Architecture

8.1 Architecture Overview and Design Approach

Château Collective uses a secure layered monolithic Flask web architecture deployed on the provided AWS VM, with Jinja2 templates, Bootstrap, SQLite, SQLAlchemy, Flask session management and Werkzeug password hashing. The system is kept as one deployable web application to reduce deployment complexity while separating the internal design into presentation, security controls, business services, persistence, file storage, audit logging and DevSecOps testing. Security is built into the main marketplace workflows, including registration, login, seller onboarding, listing management, image upload, product search, cart management, purchase commitment, simulated checkout, shipment tracking, review submission, dispute handling and administrator enforcement. The design applies defence-in-depth through HTTPS, reverse proxy controls, secure sessions, CSRF protection, RBAC, object ownership checks, server-side workflow validation, secure file validation, audit logging, abuse monitoring and protected storage. This ensures that the architecture directly addresses the identified threats and attack surfaces, including account takeover, fake registration, IDOR, seller fraud, malicious upload, search abuse, price tampering, workflow bypass, false status updates, fake reviews, invalid disputes, admin abuse and audit log tampering.

8.2 High-Level Architecture

Château Collective is structured into five layers: Client, Edge Protection, Web Application, Business Service, and Data and Storage.

The Client Layer covers guest, buyer, seller and administrator browsers. The Edge Protection Layer handles HTTPS/TLS, reverse proxy controls, security headers, rate limiting and bot filtering. The Web Application Layer contains Flask routes, Jinja2 templates, sessions, CSRF protection, authentication, RBAC and ownership checks. The Business Service Layer enforces marketplace rules for users, sellers, listings, uploads, carts, orders, workflow validation, reviews, disputes, admin actions, audit logging and backup/recovery. The Data and Storage Layer protects the SQLite database, uploaded images, audit logs, security logs and backups. This layered design supports defence-in-depth by ensuring requests pass through transport, access control, workflow validation and protected storage controls before sensitive actions are allowed. The detailed high-level architecture table and diagrams are provided in [Appendix G-1](#).

The high-level architecture separates external access, request filtering, web application logic, business rules and storage. This supports defence-in-depth because a failure in one control does not automatically expose the entire system. For example, even if a malicious user reaches a Flask route, the request must still pass session validation, CSRF validation, RBAC, ownership checks, server-side workflow validation and audit logging.

The architecture also reduces attack surface by avoiding unnecessary external services. The system does not include a real payment gateway because the checkout is simulated. It does not expose uploaded files directly as executable content. It does not expose SQLite, logs or backup

files through public web routes. This keeps the architecture aligned with the project scope while still addressing the risks identified in the threat model and attack surface analysis.

8.3 Logical Architecture

The logical architecture of Château Collective is organised into five main trust zones: External Users and Untrusted Requests, Flask Web Application, Backend Business Logic and Workflow Processing, Privileged Administrative Functions, and Protected Backend Storage.

External requests from guests, buyers, sellers, administrators and potential attackers first cross the External User Boundary, where requests are filtered, validated, authenticated and authorised through HTTPS, reverse proxy controls, security headers, rate limiting, session management, CSRF protection, RBAC and ownership checks. Valid requests then cross the Application Boundary into backend services that handle user profiles, seller applications, product listings, secure file uploads, carts, orders, workflow processing, simulated authentication status, reviews and disputes. Administrator actions are separated by the Administrative Boundary, while all application and admin services must cross the Data Storage Boundary before accessing SQLite records, uploaded images, audit logs, security logs and backups.

The rationale for this design is to separate untrusted input, application handling, business rules, privileged administration and protected storage into clear logical areas. This makes security controls easier to enforce at the correct boundary: external requests are checked before reaching application logic, business workflows are validated server-side, administrator actions are restricted and logged, and backend data is accessed only through controlled services. The design directly addresses the identified attack surfaces and threats, including account takeover, IDOR, malicious uploads, price tampering, workflow bypass, fake reviews, dispute abuse, admin misuse and audit log tampering. It also supports OWASP defence-in-depth and least-privilege principles, NIST SSDF secure design practices, and OSA security requirement categories. The detailed logical trust zones table, logical architecture diagram and trust-boundary control mapping are provided in [Appendix G-2](#).

8.4 Physical Architecture

The physical architecture of Château Collective is deployed on the provided AWS VM, where external users access the web application through HTTPS. Requests first pass through the AWS security group and reverse proxy before reaching the Flask application running behind a WSGI server. The same VM hosts the SQLite database, uploaded image directory, audit and security logs, backup storage, and protected environment configuration. Public web access is limited to the application entry point, while sensitive storage areas such as the database, uploads, logs, backups and configuration files are kept outside public web routes with restricted

permissions. The detailed physical deployment diagram and storage layout are provided in [Appendix G-3](#).

This physical design keeps the deployment simple while still supporting defence-in-depth, least privilege and secure storage separation. It reduces unnecessary infrastructure exposure by avoiding additional external services that are not required for the system, while still protecting the main attack surfaces identified in the threat model and attack surface analysis. The use of HTTPS, restricted SSH, reverse proxy controls, protected file locations, safe upload storage, audit logging, backups, GitHub Actions, dependency scanning, SAST and OWASP ZAP DAST supports OWASP secure design principles, NIST SSDF practices and OSA security requirement categories.

8.5 Component Architecture

The component architecture separates Château Collective into presentation, security, business service, persistence and DevSecOps components.

The presentation components handle public pages, authentication pages, buyer dashboards, seller dashboards and administrator pages. The security components provide authentication, administrator verification, session management, RBAC, ownership checks, CSRF protection, input validation, output encoding, security headers and abuse monitoring. The business service components enforce marketplace functions such as user management, seller application review, listing management, secure file upload, cart handling, order workflow validation, simulated authentication status updates, reviews, disputes, admin enforcement, audit logging and backup/recovery. The persistence components manage SQLite records, uploaded images, audit logs, security events and backup storage. The detailed component diagram and component responsibility table are provided in [Appendix G-4](#).

This component design supports separation of responsibility while keeping the system manageable as a single Flask application. It ensures that security controls are applied before business actions are processed and that sensitive actions such as listing approval, file upload, workflow updates, dispute handling and admin enforcement are handled through dedicated services. The design supports OWASP secure design principles, NIST SSDF practices and OSA security requirement categories by enforcing least privilege, defence-in-depth, secure workflow validation, protected storage, auditability and secure development testing.

9. Security Design

9.1. Security Design Principles

The following principles govern all security-related design decisions within Château Collective. With reference to [Appendix H-1](#), each principle is enforced across multiple system layers to ensure consistent protection.

9.2 Mitigating Threats (STRIDE)

After the Level 0 DFD was created, Microsoft Threat Modeling Tool generated STRIDE-based threats. The generated threats were reviewed based on their relevance to Château Collective's use cases, misuse cases, and identified risks. Repeated tool-generated threats were grouped into the main STRIDE categories.

With reference to [Appendix H-2](#) for STRIDE categories table.

9.3. Key Security Design Decisions

The following design decisions define how security is built into the system's core behaviour.

9.3.1. Do Not Trust Frontend Values

During checkout or order processing, the system shall not trust the following values from the client:

- product_price
- total_amount
- payment_status
- authentication_status
- user_role
- seller_id
- order_status

These values must be calculated, verified, or updated on the server side.

9.3.2. Use Deny-by-Default Access Control

Every protected endpoint must verify all of the following before executing business logic:

- Is the user logged in?
- Does the user have the required role?
- Does the user own this resource?
- Is the requested action allowed in the current workflow state?

9.3.3. Separate Admin Functions Clearly

Admin logic must not be mixed with standard buyer or seller logic. All administrative operations shall use dedicated routes, for example:

- /admin/users
- /admin/users/{id}/suspend
- /admin/orders/{id}/authentication
- /admin/orders/{id}/status
- /admin/disputes/{id}

9.3.4. Simulate Payment

The project shall not integrate a real payment gateway. Payment shall be represented through server-controlled state transitions:

Payment status: Pending → Paid → Failed → Refunded

The system shall only allow valid transitions. Specifically:

- The buyer can initiate a simulated payment.
- The system can mark a simulated payment as paid.
- The seller cannot manually mark a payment as paid.
- The buyer cannot change payment status through request tampering.

This provides security depth without real payment risk.

9.3.5. Authentication Workflow for Second-Hand Items

For second-hand items, authentication shall occur only after buyer commitment. The required workflow is:

- Available
- Buyer commits to purchase
- Seller ships item to Château Collective
- Item under in-house authentication
- Admin marks item as authenticated or rejected
- If authenticated: transaction proceeds
- If rejected: transaction is cancelled or disputed

This workflow demonstrates secure transaction control, business-logic validation, and admin-controlled state transitions.

9.3.6. Security Event Alerting (NFSR-20)

In addition to audit logging, the system shall generate administrator-visible alerts for the following security events:

- Repeated failed login attempts
- Repeated access control failures
- Excessive or rate-limited requests
- Suspicious admin actions
- Abnormal bot-like traffic patterns

Alerting provides active visibility into potential attacks, complementing the passive record kept by audit logs.

9.3.7. Enforce Security Headers (FSR-20)

The system shall configure secure HTTP response headers on all public pages, authenticated user pages, admin pages, and API responses to reduce browser-based attacks. Required headers include:

- Content-Security-Policy — restricts sources for scripts, styles, and other resources to prevent XSS.
- X-Frame-Options — prevents the application from being embedded in iframes (clickjacking protection).

- X-Content-Type-Options — disables MIME-type sniffing by the browser.
- Referrer-Policy — controls how much referrer information is included with requests.

9.4 Workflow State Design

The order and authentication workflow is controlled by a server-side workflow state machine. Client-submitted workflow values are not trusted. The system calculates and validates all state transitions on the server side. The workflow states designed for the system are: Available, Committed, Awaiting Shipment, Shipped, Under Authentication, Authenticated, Rejected, Sold. The detailed workflow state table is provided in [Appendix H-3](#).

Workflow Security Rules

1. A buyer can commit to purchase only an approved, public and available item.
2. The committed price must be copied from the server-side product listing record.
3. The buyer cannot submit their own price, order status or authentication status.
4. Sellers can update shipment status only for orders linked to their own listings.
5. Sellers cannot mark their own items as authenticated.
6. Only administrators can update simulated authentication status.
7. Simulated checkout can complete only after required workflow conditions are satisfied.
8. Invalid workflow transitions are rejected and logged.
9. Important workflow changes are recorded in audit logs with actor identity, timestamp, old state, new state and target record.

9.5 Class Diagram and Database Model

The class diagram and database model are designed around the main business entities in Château Collective, including users, profiles, seller applications, product listings, uploaded files, carts, orders, shipments, authentication reviews, reviews, disputes, audit logs, security events and backup records. Each sensitive or private record is linked to the relevant user, buyer, seller, order or administrator so that RBAC and object ownership checks can be enforced at application level. The model also includes workflow-related entities such as AuthenticationReview and OrderStatusHistory to support the simulated authentication process, order state tracking and accountability. The detailed class diagram is provided in [Appendix H-4](#).

The rationale for this design is to make security controls enforceable through the data model, not only through route-level checks. Foreign-key relationships support referential integrity, while ownership fields such as buyer_id, seller_id, listing_id and order_id help prevent IDOR and unauthorised record access. Security-related tables such as AuditLog, SecurityEvent, UploadedFile and BackupRecord support investigation, upload validation, abuse monitoring and recovery. This design supports OWASP secure design principles, NIST SSDF practices and OSA security requirement categories by strengthening access control, workflow integrity, auditability and protected storage.

Appendix A

Appendix A-1 - Secure Functional Requirements

Authentication & User Management

SFR-01 - Secure User Registration

The system shall allow guests to register an account using name, email address, phone number, and password. The system shall validate all registration inputs and shall not allow duplicate email accounts, invalid email formats, or weak passwords.

SFR-02 - Secure User Authentication

The system shall allow registered buyers, sellers, and administrators to log in and log out. The system shall not allow access to authenticated functions without a valid login session.

SFR-03 - Secure Profile Management

The system shall allow authenticated buyers and sellers to update only their own profile information. The system shall not allow users to view or modify another user's private profile details.

SFR-04 - Secure Seller Application

The system shall allow regular users to apply to become verified sellers. Seller application details shall only be viewable by the applicant and authorised administrators.

Catalog & Product Management

SFR-05 - Secure Product Browsing

The system shall allow guests and users to browse public product listings. Private seller information, internal listing flags, and admin review notes shall not be exposed on public product pages.

SFR-06 - Secure Product Search and Filtering

The system shall allow users to search and filter listings by category, brand, condition, and price. The system shall not allow unvalidated users to search inputs to prevent injection attacks and abnormal queries.

SFR-07 - Secure Listing Creation and Editing

The system shall allow authenticated sellers to create and edit only their own listings. The system shall not allow sellers to edit, delete, or manipulate listings belonging to other sellers.

Order, Cart & Transaction Workflow

SFR-08 - Secure Cart Management

The system shall allow buyers to manage their own shopping cart. The system shall not allow buyers to be able to view or modify another buyer's cart.

SFR-09 - Secure Purchase Commitment

The system shall allow buyers to commit to purchasing an item. The system shall verify that the item is available and that the buyer is authorised before creating the purchase commitment.

SFR-10 - Secure Shipment Workflow Tracking

The system shall allow sellers to update shipment status only for orders involving their own

listings. The system shall not allow sellers to be able to update shipment records for unrelated orders.

SFR-11 - Secure Simulated Checkout

The system shall complete the simulated checkout only after all required workflow status checks are satisfied. The system shall not mark an order as completed if required order, shipment, or compliance checks are missing.

Admin Platform Operations

SFR-12 - Secure Admin Dashboard

The system shall provide an admin dashboard only to authenticated administrators. The system shall not allow Non-admin users to be able to access administrative functions.

SFR-13 - Secure User Suspension

The system shall allow administrators to suspend suspicious accounts. All suspension actions shall be logged with administrator identity, timestamp, action type, and affected account.

SFR-14 - Secure Listing Monitoring

The system shall allow administrators to review suspicious or reported listings. Admin review notes and moderation decisions shall not be visible to unauthorised users.

SFR-15 - Secure Order and Workflow Updates

The system shall allow administrators to update order and workflow states based on compliance requirements. All manual workflow changes shall be logged for accountability.

Review & Dispute Management

SFR-16 - Secure Purchase-Based Review System

The system shall allow buyers to submit ratings and reviews only for successfully completed purchases. The system shall not allow users to be able to review items they did not purchase.

SFR-17 - Secure Dispute Resolution Workflow

The system shall allow buyers to raise disputes for their own orders only. Dispute details shall only be accessible to the buyer involved, relevant seller, and authorised administrators.

Appendix A-2 - Functional Security Requirements

Authentication

FSR-01 - Administrator Authentication

The system shall require administrators to authenticate using two-factor authentication before accessing the admin dashboard.

FSR-02 - Password Security

The system shall enforce a password policy that rejects passwords that are too short, commonly used, easily guessable, or known to be compromised. Passwords shall not be accepted if they are too short, commonly used, easily guessable, or known to be compromised.

FSR-03 - Secure Password Handling

The system shall store user passwords using a secure one-way password hashing algorithm with salting and shall not store plaintext passwords.

FSR-04 - Session Management

The system shall create a secure session after successful login and terminate the session after logout or inactivity timeout.

FSR-05 - Brute Force Protection

The system shall apply progressive login throttling after repeated failed login attempts. The system shall temporarily restrict login attempts based on account, IP address, session, device fingerprint, or suspicious behaviour, while reducing the risk that attackers can deliberately lock legitimate users out of their accounts.

FSR-06 - Account Abuse Prevention

The system shall apply controls to reduce account-related abuse, including repeated registration attempts, repeated failed login attempts, fake account creation, and abnormal login behaviour. The system shall log suspicious authentication activity and shall temporarily block, challenge, or restrict suspicious account-related requests.

Authorisation

FSR-07 - Role-Based Access Control

The system shall implement role-based access control for Guest, Buyer, Seller, and Administrator roles.

FSR-08 - Buyer Access Control

The system shall ensure buyers can only access their own profile, cart, orders, reviews, and dispute records.

FSR-09 - Seller Access Control

The system shall ensure sellers can only create, edit, and manage listings and shipment updates related to their own products.

FSR-10 - Admin Access Control

The system shall ensure only administrators can suspend users, review suspicious listings, update workflow states, and resolve disputes.

Accountability

FSR-11 - Audit Logging

The system shall log important security and business actions, including login attempts, listing creation, purchase commitment, shipment updates, checkout, dispute creation, user suspension, listing monitoring, order workflow updates, and dispute resolution.

FSR-12 - Admin Action Traceability

The system shall record administrator identity, timestamp, action type, and target record for all important admin actions.

FSR-13 - Failed Event Monitoring

The system shall log failed authentication attempts and suspicious access activities.

FSR-14 - Audit Log Protection

The system shall protect audit log records from unauthorised modification or deletion and shall restrict access to audit logs to authorised administrators or system processes.

Data Protection

FSR-15 - Protection of Private Records

The system shall prevent users from viewing private records belonging to other users, including profile details, cart contents, order history, review eligibility, and dispute details.

FSR-16 - Data Minimisation

The system shall only collect personal data required for registration, profile management, seller application, order processing, review handling, and dispute management.

FSR-17 - Secure Simulated Payment Data Handling

The simulated checkout shall not collect or store real payment card numbers, CVV values, or banking credentials. Any payment-related input used for demonstration shall be clearly simulated and shall not represent real payment data.

Encryption and Communication Protection

FSR-18 - Transport Encryption

The system shall encrypt all client-server communications using HTTPS/TLS.

FSR-19 - Stored Data Protection

The system shall protect sensitive stored data, including database records, uploaded files, and backup files, using encryption at rest or equivalent secure storage access controls.

FSR-20 - Security Header Enforcement

The system shall configure secure HTTP response headers for all public pages, authenticated user pages, admin pages, and API responses to reduce browser-based attacks.

FSR-21 - CSRF Protection

The system shall protect authenticated state-changing requests using CSRF protection or equivalent anti-forgery controls. This shall apply to profile updates, listing creation and editing, cart updates, purchase commitment, checkout, shipment updates, review submission, dispute submission, and admin actions.

Integrity

FSR-22 - Secure Transaction Business Rules

The system shall enforce cart, purchase commitment, shipment, checkout, review, and dispute business rules on the server side. The system shall not rely on client-side values for price, item availability, order ownership, workflow status, review eligibility, or dispute eligibility.

The system shall reject any request that attempts to bypass the approved order workflow, modify another user's transaction record, commit to an unavailable item, submit a review without a completed purchase, or raise a dispute for an unrelated order.

Availability

FSR-23 - Application-Layer DDoS and Excessive Request Protection

The system shall provide application-layer excessive request protection using application-level, reverse-proxy, or network-level controls to detect, throttle, reject, or temporarily block abnormal traffic that may degrade system availability.

FSR-24 - Bot and Automated Traffic Filtering

The system shall detect, restrict, and block abnormal automated traffic patterns that may affect system availability, data confidentiality, or business workflow integrity.

The system shall apply traffic filtering controls based on request frequency, IP address, user account, session, user agent, and endpoint behaviour. The system shall restrict or block automated scraping, credential-stuffing attempts, excessive product search requests, repeated image upload attempts, fake registration attempts, and abnormal checkout or dispute submissions.

Backup and Recovery

FSR-25 - Backup Functionality

The system shall support backup of important user, product, order, review, dispute, and admin operation data.

FSR-26 - Recovery Functionality

The system shall support restoration of backup data without loss of critical user, product, order, review, dispute, and admin operation records.

Appendix A-3 - Non-Functional Security Requirements

Confidentiality

NFSR-01 - Protection of Personal Data

The system shall prevent unauthorised disclosure of personal data collected during registration, profile management, seller applications, orders, reviews, and disputes. During access control and negative testing, unauthorised users shall not be able to view, retrieve, or modify personal data belonging to other users.

NFSR-02 - Access Restriction for Private Records

Unauthorised attempts to access another user's profile, cart, order history, review eligibility, or dispute details shall be blocked during access control and negative testing.

NFSR-03 - Data Minimisation Compliance

Registration, profile, seller application, and dispute forms shall only request personal information required for the stated business purpose. This shall be verified through form review and requirements traceability review.

NFSR-04 - Data Encryption in Transit Coverage

All tested client-server communication involving login, registration, profile management, seller application, cart, checkout, review, dispute, and admin functions shall be transmitted over HTTPS using TLS 1.2 or above, with no sensitive data transmitted over plaintext HTTP.

NFSR-05 - Stored Data Protection

Sensitive stored data, including user personal data, seller application data, dispute records, and backup files, shall remain unreadable or inaccessible to unauthorised users. Stored data and backup files shall not be accessible through public web routes or unauthorised server accounts, and shall be protected through encryption at rest or equivalent secure storage controls.

Integrity

NFSR-06 - Data Integrity

The system shall maintain accurate and consistent records for users, seller applications, listings, carts, orders, reviews, disputes, and admin actions. During integration testing, database records and displayed information shall not show unexplained mismatches after create, update, delete, and workflow operations.

NFSR-07 - Transaction Integrity

Cart updates, purchase commitments, shipment updates, checkout actions, review submissions, and dispute submissions shall not corrupt existing system data when an error occurs. Order workflow states shall only transition according to approved business rules.

NFSR-08 - Admin Action Integrity

User suspension, listing monitoring, order workflow updates, and dispute resolution records shall remain accurate and traceable. During audit log review, each important admin action shall be linked to an administrator identity, timestamp, action type, and target record.

Availability

NFSR-09 - System Availability

The system shall achieve at least 99% uptime during the project testing and demonstration period, based on repeated access testing or uptime monitoring.

NFSR-10 - Concurrent User Availability

The system shall support at least 50 concurrent users in the test environment without major errors, failed requests, or response times exceeding the defined performance thresholds.

NFSR-11 - Backup and Recovery Availability

The system shall restore critical data from a database backup within one hour during recovery testing.

NFSR-12 - Excessive Request Resilience

Under excessive request testing on high-risk endpoints, including registration, login, product search, image upload, cart update, purchase commitment, checkout, review submission, dispute submission, and admin login, the system shall delay, reject, or temporarily block requests that exceed the configured threshold without crashing the system or preventing normal users from accessing unaffected functions.

Performance

NFSR-13 - Secure Page Response Time

Product browsing, product details, cart, checkout, profile, and admin pages shall load within 3 seconds under normal test load.

NFSR-14 - Secure Search Performance

Search and filtering results shall be returned within 2 seconds for the normal project dataset size under normal test load.

NFSR-15 - Secure Transaction Processing

Cart updates, purchase commitment, shipment updates, and checkout actions shall complete within 3 seconds under normal test load.

NFSR-16 - Secure Image Upload Performance

Valid product image uploads that meet the allowed file type, file size, file extension, and MIME type restrictions shall complete within 5 seconds under normal network and server conditions.

Auditability and Observability

NFSR-17 - Audit Log Integrity

Audit logs shall be protected against unauthorised modification or deletion. During access control testing, non-admin users shall not be able to alter or delete audit logs, and administrative access to audit logs shall only occur through authorised system functions.

NFSR-18 - Security Log Quality

Security logs for failed login attempts, listing creation, purchase commitment, shipment updates, checkout, dispute creation, and admin actions shall include, where applicable, timestamp, event type, actor identity, action performed, source information, and target record.

NFSR-19 - Business Record Retention

Completed orders, reviews, disputes, and admin actions shall remain retrievable after workflow completion during the project lifecycle for accountability and audit review.

NFSR-20 - Security Event Alerting

During security testing, repeated failed login attempts, repeated access control failures, excessive requests, suspicious admin actions, and abnormal bot-like traffic patterns shall generate administrator-visible alerts or notifications.

Error Handling**NFSR-21 - Secure Error Messages**

The system shall not reveal stack traces, database schema details, server paths, or internal implementation details in error responses. The system shall display controlled error messages to users while recording sufficient server-side diagnostic details in protected logs for debugging and investigation.

Appendix A-4 - Secure Development Requirements

Secure Coding

SDR-01 - Input Validation

All user inputs, including registration details, profile updates, product listings, search filters, image uploads, checkout actions, reviews, and disputes, shall be validated before processing.

SDR-02 - Output Encoding

User-generated content such as product descriptions, seller information, reviews, and dispute messages shall be encoded before display to reduce cross-site scripting risks.

SDR-03 - SQL Injection Prevention

The system shall use parameterised queries or secure ORM methods for all database operations.

SDR-04 - Secure File Upload Handling

Product image uploads shall be restricted by file type, file size, file extension, and verified MIME type. Uploaded files shall be renamed using system-generated filenames, stored in a non-executable upload directory, and associated only with the authenticated seller's own listing. Invalid, oversized, or potentially unsafe files shall be rejected.

Authentication and Password Development

SDR-05 - Password Storage Implementation

Developers shall implement secure password hashing and shall ensure plaintext passwords are never stored or logged.

SDR-06 - Session Security Implementation

Developers shall ensure sessions are invalidated after logout and protected against unauthorised reuse.

Testing

SDR-07 - Access Control Testing

The development team shall test that buyers, sellers, admins, and guests can only access functions and records permitted by their roles. The test shall include direct URL manipulation, object ID manipulation, and attempts to access another user's profile, cart, order, listing, review, and dispute records.

SDR-08 - Negative Testing

The development team shall test failed registration, listing creation, image upload, cart update, checkout, shipment update, review submission, and dispute submission to ensure errors do not corrupt system data.

SDR-09 - Audit Log Testing

The development team shall verify that security-relevant actions and admin actions are correctly recorded in system logs, and that unauthorised users cannot modify or delete audit log records.

SDR-10 - Backup and Restore Testing

The development team shall test database backup and restoration to confirm critical records can be recovered.

Secure Deployment

SDR-11 - Secrets Management

The system shall securely manage application secrets and shall not store sensitive credentials directly in source code or public repositories.

SDR-12 - Secure VM and Repository Deployment

The deployed system shall not expose source code, Git metadata, environment files, database files, backup files, private keys, or debug interfaces through the public web server. SSH access shall use key-based authentication and administrative access shall be restricted to authorised team members only.

SDR-13 - Secure Storage Deployment Configuration

The deployed system shall ensure database files, uploaded files, and backup files are not placed in public web directories and are only accessible to authorised server accounts or deployment processes.

SDR-14 - Server Hardening and Public Exposure Control

The deployed server shall expose only required public services, such as HTTPS for the web application. Administrative services such as SSH, database access, debug interfaces, and internal management tools shall be restricted to authorised users and shall not be publicly accessible without protection.

SDR-15 - Network Traffic Filtering Configuration

The deployed system shall configure server-side, reverse-proxy, firewall, or cloud security group rules to restrict abnormal automated traffic, suspicious user agents, repeated failed access attempts, excessive requests, and access to sensitive paths. The configuration shall be reviewed and tested before deployment.

Review and Maintenance

SDR-16 - Secure Code Review

Source code involving authentication, authorisation, order workflow, admin actions, file uploads, reviews, and disputes shall undergo review before release.

SDR-17 - Dependency Review

Third-party libraries and packages used by the system shall be reviewed to avoid known vulnerable dependencies.

SDR-18 - Browser Compatibility Testing

The web application shall be tested on the latest versions and last two major versions of Google Chrome, Microsoft Edge, and Mozilla Firefox to ensure consistent and secure behaviour across browsers.

SDR-19 - Logging and Error Handling Review

Developers shall ensure error messages are controlled and do not expose sensitive technical details, database errors, stack traces, or private user information.

Appendix B

Use Case	Main Misuse Case	Main Security Focus
UC-01 Account and Profile Management	Account Takeover, Fake Registration and Profile IDOR	Authentication, session management, access control, accountability
UC-02 Seller Onboarding and Listing Management	Fraudulent Seller, Counterfeit Listing and Malicious Upload Abuse	Seller verification, listing approval, file validation, ownership control
UC-03 Product Discovery	Search Injection, Listing Enumeration and Scraping Abuse	Input validation, public data restriction, rate limiting, bot filtering
UC-04 Cart, Purchase Commitment and Simulated Checkout	Price Tampering, Order Ownership Bypass and Workflow Skipping	Server-side business rules, transaction integrity, state enforcement
UC-05 Shipment and Order Status Tracking	False Shipment Update and Unauthorised Workflow State Change	Role restriction, state transition validation, workflow accountability
UC-06 Review and Dispute Management	Fake Review, Invalid Dispute and Dispute Abuse	Purchase verification, order ownership, abuse monitoring
UC-07 Admin Monitoring and Enforcement	Admin Account Takeover, Abusive Admin Action and Audit Log Tampering	MFA, RBAC, least privilege, protected audit logs

Table B1: Summary of Use Case to Misuse Case Mapping

Appendix C

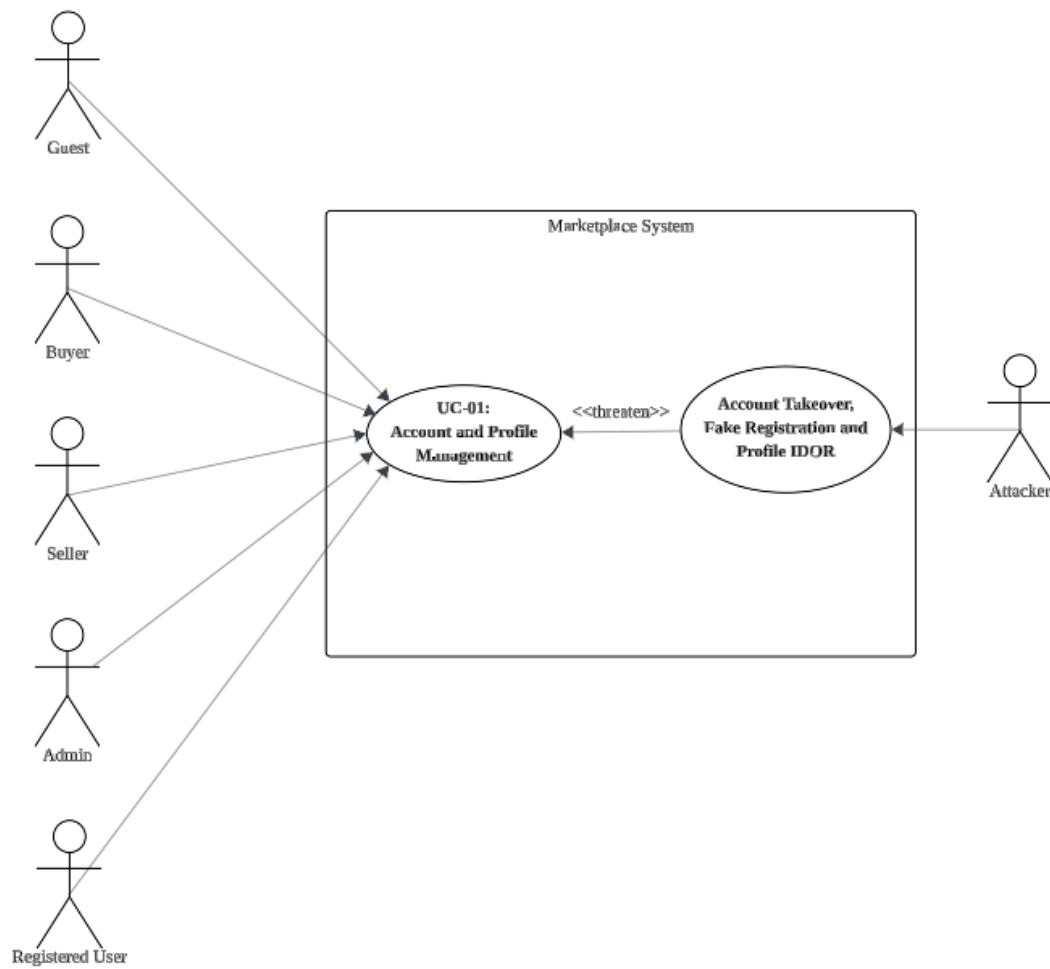


Figure C1: UC-01 Use Case Diagram

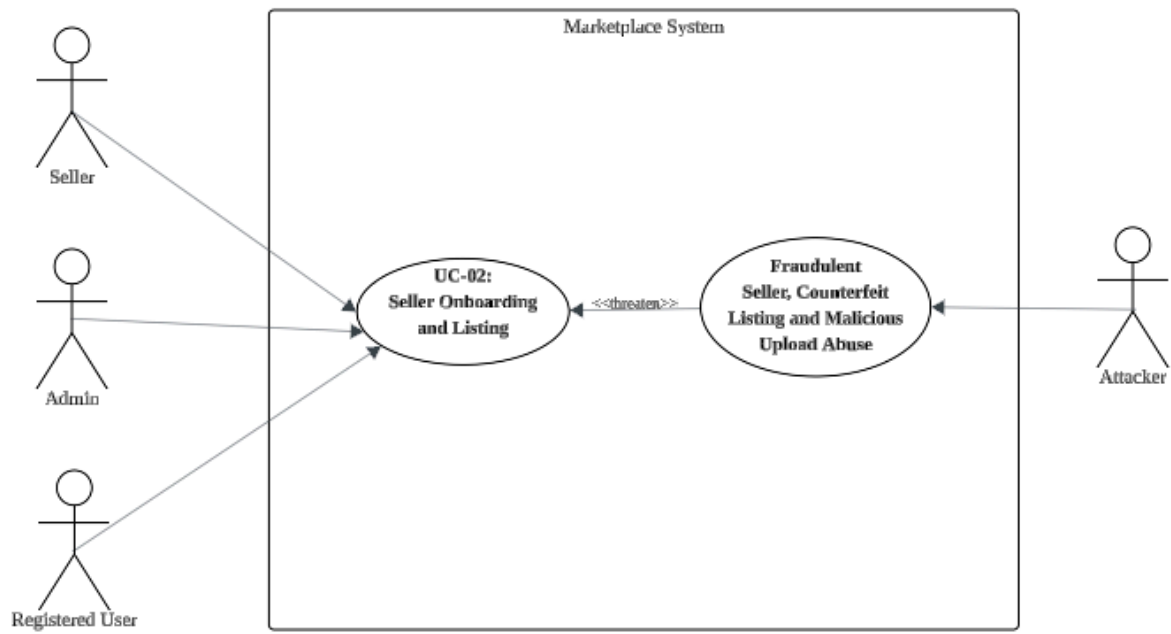


Figure C2: UC-02 Use Case Diagram

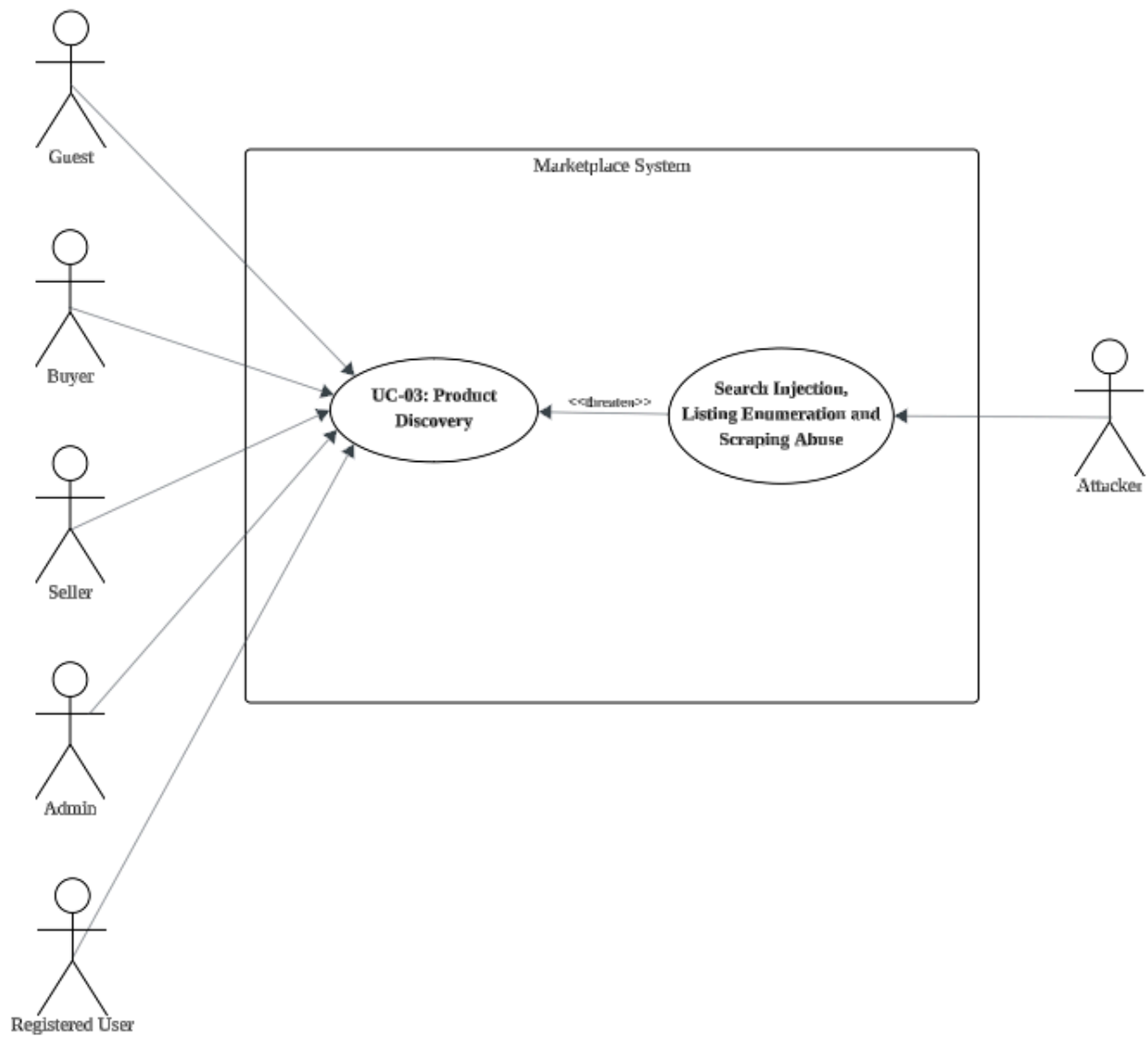


Figure C3: UC-03 Use Case Diagram

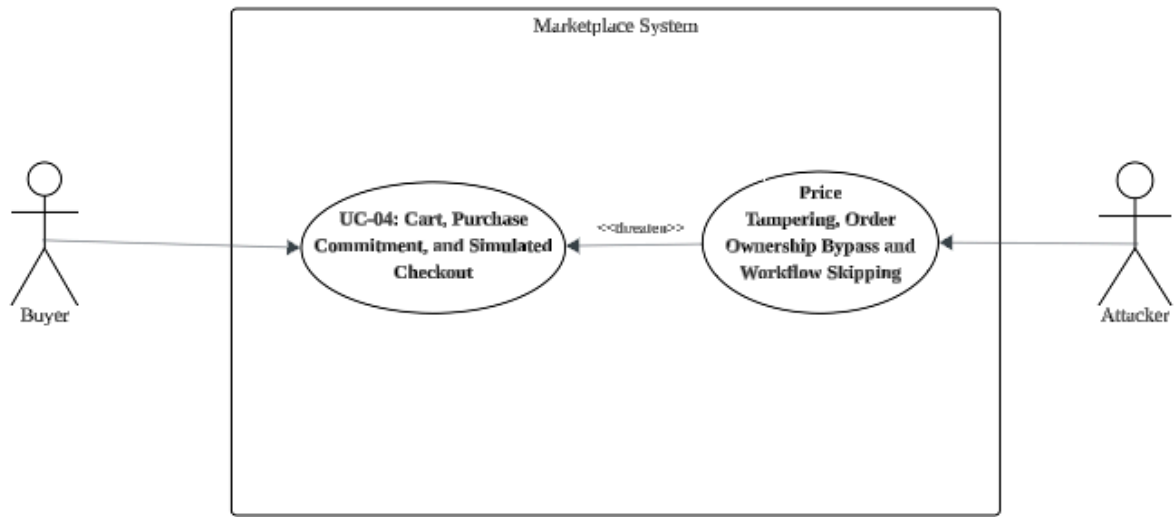


Figure C4: UC-04 Use Case Diagram

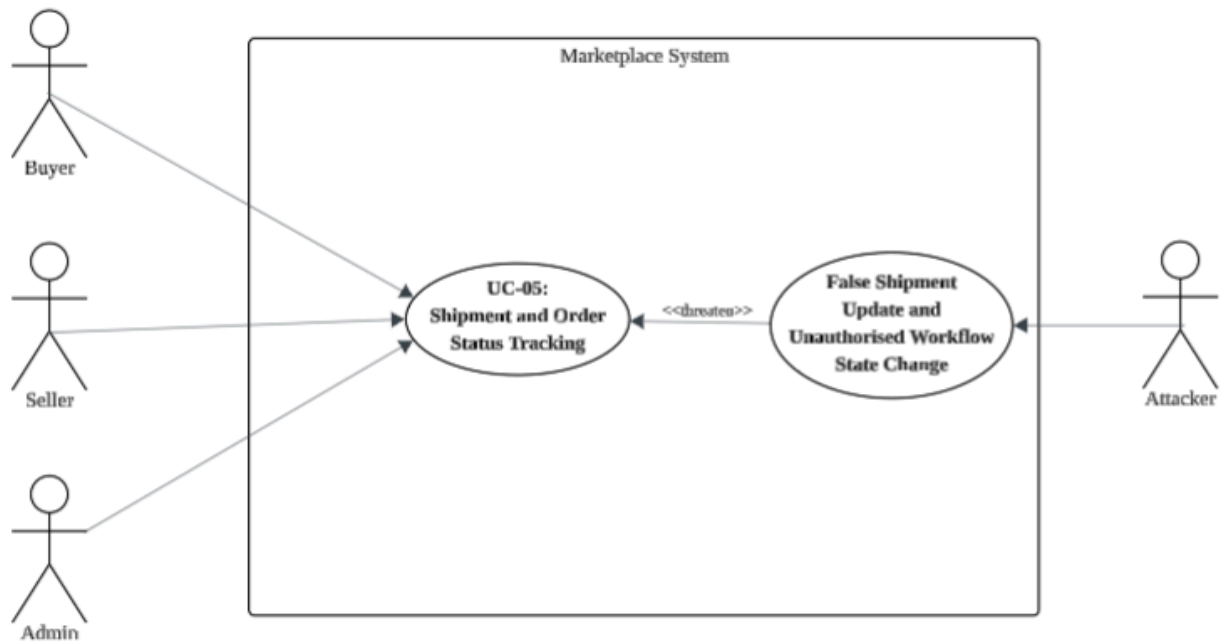


Figure C5: UC-05 Use Case Diagram

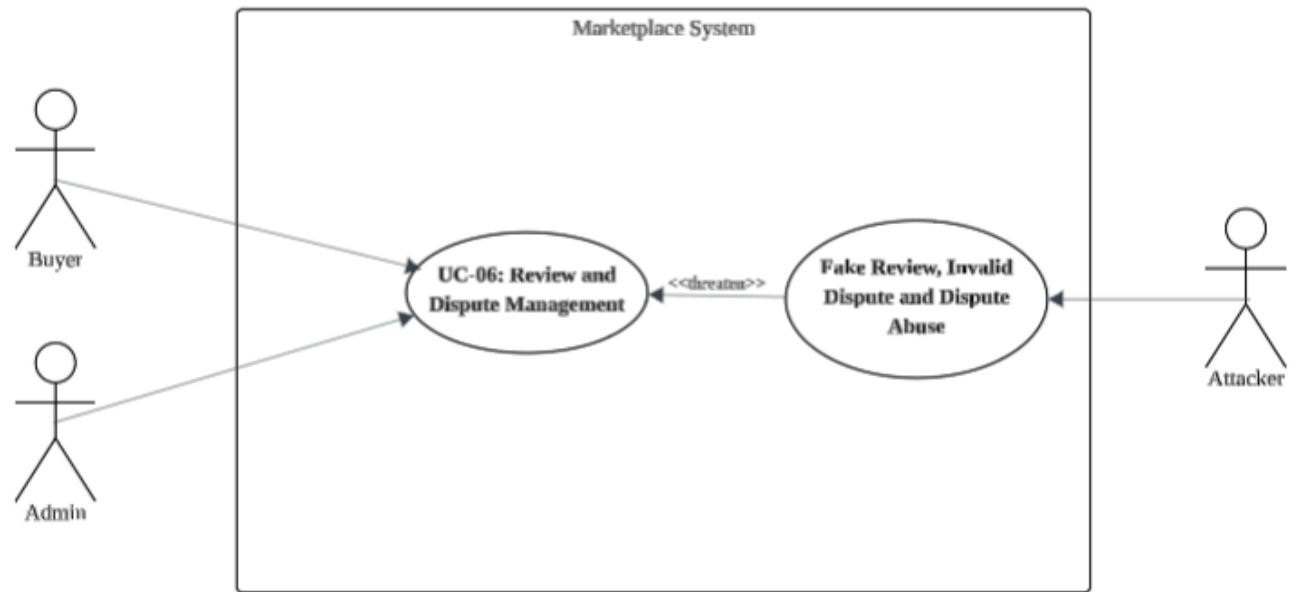


Figure C6: UC-06 Use Case Diagram

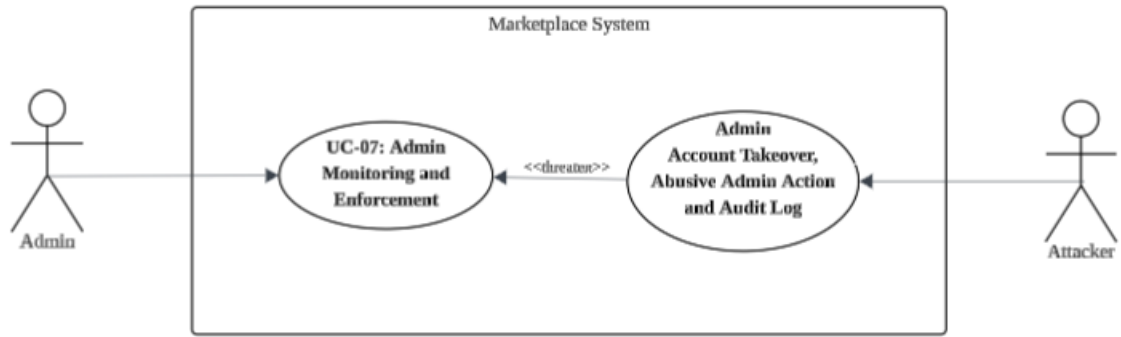


Figure C7: UC-07 Use Case Diagram

Appendix D

Risk ID	Risk Name	Description	Likelihood	Impact	Criticality
R-01	Brute Force / Credential Stuffing	Attacker repeatedly attempts login using common passwords or credential dumps from other breaches to gain account access	High	High	High
R-02	Sensitive Data / PII Exposure	User personal data (email, phone number, address) is exposed due to misconfiguration, insecure storage, or a breach.	Medium	High	High
R-03	Injection Attacks (SQLi / XSS)	Attacker injects malicious SQL or scripts through search fields, listing forms, or review inputs, compromising the database or hijacking sessions.	Medium	High	High
R-04	Unauthorized Admin Access / Privilege Escalation	Attacker compromises admin credentials or exploits broken access control to gain full platform control.	Low	High	Medium
R-05	Malicious File Upload	A rogue seller uploads files disguised as product images that contain malware, scripts, or oversized payloads.	Medium	High	High
R-06	Fake Account Registration and Platform Abuse	Attacker creates bulk fake accounts (buyers or sellers) to manipulate listings, reviews, or platform metrics.	High	Medium	High
R-07	Fraudulent Seller Onboarding	Bad actor submits falsified credentials or identity documents to gain verified seller status and conduct fraud.	Medium	High	High
R-08	False Shipment Status /	A rogue seller marks an order as shipped without	Medium	High	High

Risk ID	Risk Name	Description	Likelihood	Impact	Criticality
	Delivery Fraud	dispatching it, triggering payment release or dispute complications.			
R-09	Session Hijacking / Token Theft	Attacker steals a valid session token (e.g. via XSS or network interception) and impersonates an authenticated user.	Medium	High	High
R-10	Review Manipulation	Buyers or colluding parties post fake or ineligible reviews to artificially inflate or damage seller reputation.	High	Medium	High
R-11	Cart and Checkout Tampering	Attacker intercepts and modifies cart requests to alter prices, quantities, or bypass purchase validation rules.	Medium	High	High
R-12	Catalog Scraping and Data Harvesting	Automated bots systematically extract all product listings, pricing, and seller information for competitive misuse or resale.	High	Medium	High
R-13	Insider Threat / Compromised Admin	A legitimate admin abuses elevated privileges to suspend accounts unfairly, alter order records, or leak platform data.	Low	High	Medium
R-14	Frivolous Dispute Abuse	Dishonest buyers submit baseless disputes repeatedly to delay fulfilment, pressure sellers, or exploit refund workflows.	Medium	Medium	Medium
R-15	Denial of Service via Search / API Flooding	Attacker floods the search, filter, or checkout endpoints with excessive requests, degrading performance for all users.	Medium	Medium	Medium
R-16	Improper	Admin or seller forcibly	Low	Medium	Low

Risk ID	Risk Name	Description	Likelihood	Impact	Criticality
	Workflow State Manipulation	overrides order or shipment state transitions outside of permitted rules, creating inconsistent records.			

Table D1: Potential Risks

Appendix E

Appendix E-1 - System Data Flow

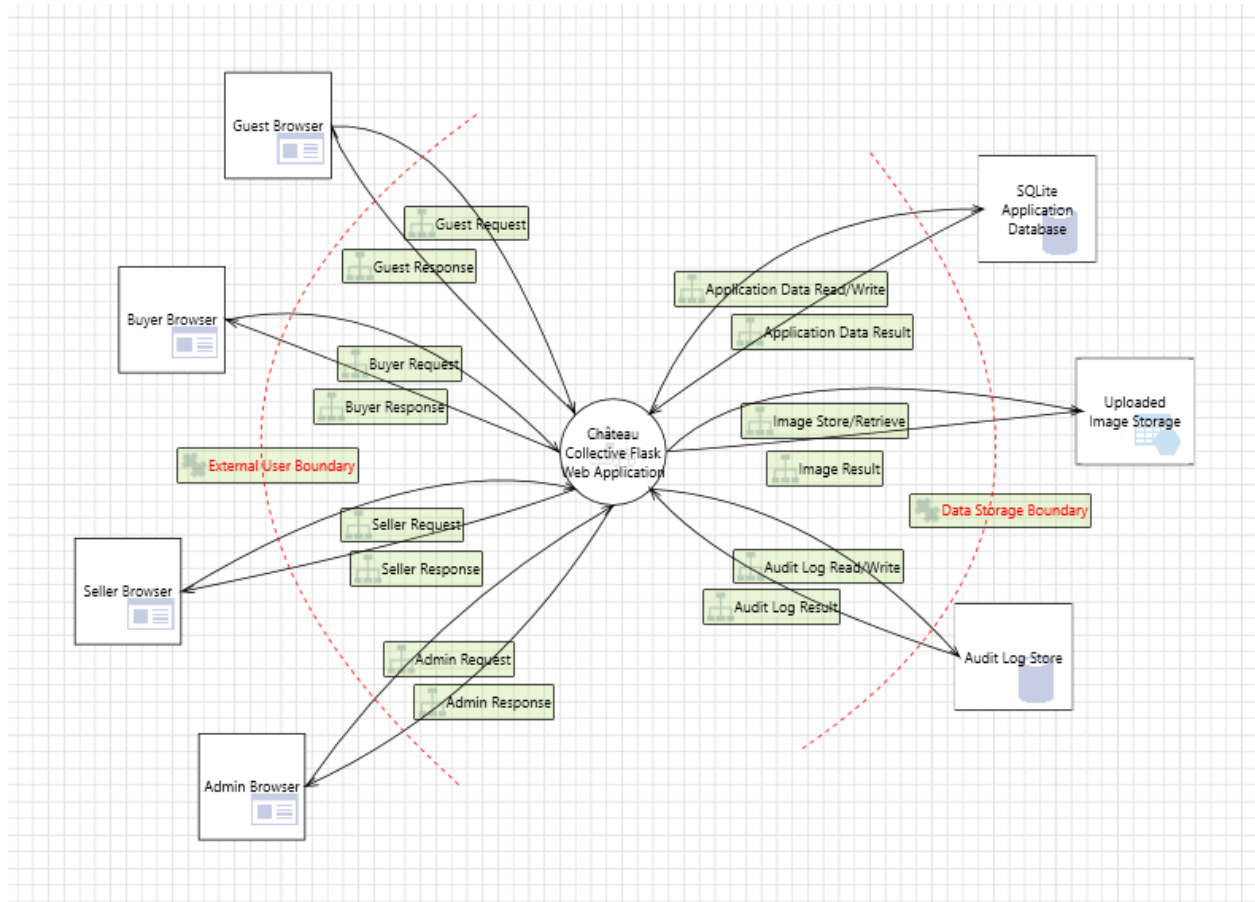


Figure E-1.1: Level 0 Data Flow Diagram

DFD Element	Description
External Entities	Guest Browser, Buyer Browser, Seller Browser, Administrator Browser
Main Process	Château Collective Flask Web Application
Data Stores	SQLite Application Database, Uploaded Image Storage, Audit Log Store
Main Data Flows	User requests and responses, application data read/write, image store/retrieve, audit log read/write
Trust Boundaries	External user boundary between browsers and the web application; data storage boundary between the web application and backend storage; administrator privilege boundary for admin-only functions

Table E-1.1: DFD Table

Appendix E-2 - Threat Modelling

Threat List												
ID	Diagram	Changed By	Last Modified	State	Title	STRIDE Category	Description	Justification	Interaction	Possible Mitig	Severity	SDL Phase
0	Diagram 1	DESKTOP-MOG8	10/6/2026 12:42:1	Mitigated	An adversary car	Denial of Service	Failure to restrict	Mitigated through	Guest Request	Ensure that auth	High	Implementation
1	Diagram 1		Generated	Not Started	An adversary ma	Elevation of Privi	Failure to restrict		Guest Request	Ensure that admi	High	Implementation
2	Diagram 1		Generated	Not Started	An adversary car	Information Disc	An adversary car		Guest Request	Do not expose se	High	Implementation
3	Diagram 1	DESKTOP-MOG8	10/6/2026 12:43:1	Needs Investigat	An adversary ma	Information Disc	An adversary ma	Mitigated through	Guest Request	Ensure that the a	High	Implementation
4	Diagram 1		Generated	Not Started	An adversary ma	Information Disc	An adversary ma		Guest Request	Ensure that sensi	High	Implementation
5	Diagram 1		Generated	Not Started	An adversary car	Information Disc	Robots.txt is ofte		Guest Request	Ensure that admi	Medium	Implementation
6	Diagram 1		Generated	Not Started	An adversary car	Information Disc	An adversary ma		Guest Request	Applications ava	High	Implementation
7	Diagram 1		Generated	Not Started	An adversary car	Information Disc	An adversary car		Guest Request	Do not expose se	High	Implementation
8	Diagram 1		Generated	Not Started	An adversary ma	Information Disc	An adversary ma		Guest Request	Ensure that sensi	High	Implementation
9	Diagram 1		Generated	Not Started	Attacker can den	Repudiation	Proper logging o		Guest Request	Ensure that audit	Medium	Implementation
10	Diagram 1	DESKTOP-MOG8	10/6/2026 12:42:1	Mitigated	An adversary car	Spoofing	The session cook	Mitigated by strc	Guest Request	Set up session fo	High	Implementation
11	Diagram 1	DESKTOP-MOG8	10/6/2026 12:43:1	Mitigated	An adversary car	Spoofing	The session cook	Mitigated by strc	Guest Request	Enable ValidateR	High	Implementation
12	Diagram 1		Generated	Not Started	An adversary car	Spoofing	Ensure that TLS c		Guest Request	Verify X.509 certifi	High	Implementation
13	Diagram 1		Generated	Not Started	An adversary car	Spoofing	Attackers can exj		Guest Request	Explicitly disable	High	Implementation
14	Diagram 1		Generated	Not Started	Attackers can ste	Spoofing	The session cook		Guest Request	Applications ava	High	Implementation
15	Diagram 1		Generated	Not Started	An adversary car	Spoofing	Phishing is atten		Guest Request	Verify X.509 certifi	High	Implementation
16	Diagram 1		Generated	Not Started	An adversary ma	Spoofing	If proper authent		Guest Request	Consider using a	High	Design
17	Diagram 1		Generated	Not Started	An adversary car	Tampering	Website defacer		Guest Request	Implement Cont	High	Implementation
18	Diagram 1		Generated	Not Started	An attacker steal	Tampering	An attacker steal		Guest Request		High	Implementation
19	Diagram 1		Generated	Not Started	An adversary car	Tampering	SQL injection is z		Guest Request	Ensure that type-	High	Implementation
20	Diagram 1		Generated	Not Started	An adversary car	Tampering	An adversary car		Guest Request	Encrypt sections	High	Implementation
21	Diagram 1		Generated	Not Started	An adversary car	Denial of Service	Failure to restrict		Buyer Request	Ensure that auth	High	Implementation
22	Diagram 1		Generated	Not Started	An adversary ma	Elevation of Privi	Failure to restrict		Buyer Request	Ensure that admi	High	Implementation
23	Diagram 1		Generated	Not Started	An adversary car	Information Disc	An adversary car		Buyer Request	Do not expose se	High	Implementation
24	Diagram 1		Generated	Not Started	An adversary ma	Information Disc	An adversary ma		Buyer Request	Ensure that the a	High	Implementation
25	Diagram 1		Generated	Not Started	An adversary ma	Information Disc	An adversary ma		Buyer Request	Ensure that sensi	High	Implementation
26	Diagram 1		Generated	Not Started	An adversary car	Information Disc	Robots.txt is ofte		Buyer Request	Ensure that admi	Medium	Implementation
27	Diagram 1		Generated	Not Started	An adversary ma	Information Disc	An adversary ma		Buyer Request	Applications ava	High	Implementation
28	Diagram 1		Generated	Not Started	An adversary car	Information Disc	An adversary car		Buyer Request	Do not expose se	High	Implementation
29	Diagram 1		Generated	Not Started	An adversary ma	Information Disc	An adversary ma		Buyer Request	Ensure that sensi	High	Implementation
30	Diagram 1	DESKTOP-MOG8	10/6/2026 12:43:1	Mitigated	Attacker can den	Repudiation	Proper logging o	Mitigated by auc	Buyer Request	Ensure that audit	Medium	Implementation
31	Diagram 1		Generated	Not Started	An adversary car	Spoofing	The session cook		Buyer Request	Set up session fo	High	Implementation
32	Diagram 1		Generated	Not Started	An adversary car	Spoofing	The session cook		Buyer Request	Enable ValidateR	High	Implementation
33	Diagram 1		Generated	Not Started	An adversary car	Spoofing	Ensure that TLS c		Buyer Request	Verify X.509 certifi	High	Implementation
34	Diagram 1		Generated	Not Started	An adversary car	Spoofing	Attackers can exj		Buyer Request	Explicitly disable	High	Implementation
35	Diagram 1		Generated	Not Started	Attackers can ste	Spoofing	The session cook		Buyer Request	Applications ava	High	Implementation
36	Diagram 1		Generated	Not Started	An adversary car	Spoofing	Phishing is atten		Buyer Request	Verify X.509 certifi	High	Implementation
37	Diagram 1		Generated	Not Started	An adversary ma	Spoofing	If proper authent		Buyer Request	Consider using a	High	Design
38	Diagram 1	DESKTOP-MOG8	10/6/2026 12:43:1	Mitigated	An adversary car	Tampering	Website defacer	Mitigated by sen	Buyer Request	Implement Cont	High	Implementation
39	Diagram 1		Generated	Not Started	An attacker steal	Tampering	An attacker steal		Buyer Request		High	Implementation
40	Diagram 1	DESKTOP-MOG8	10/6/2026 12:43:1	Mitigated	An adversary car	Tampering	SQL injection is z	Mitigated by sen	Buyer Request	Ensure that type-	High	Implementation
41	Diagram 1		Generated	Not Started	An adversary car	Tampering	An adversary car		Buyer Request	Encrypt sections	High	Implementation
42	Diagram 1		Generated	Not Started	An adversary ma	Denial of Service	Failure to restrict		Seller Request	Ensure that auth	High	Implementation
43	Diagram 1	DESKTOP-MOG8	10/6/2026 12:43:1	Mitigated	An adversary ma	Elevation of Privi	Failure to restrict	Mitigated by role	Seller Request	Ensure that admi	High	Implementation
44	Diagram 1		Generated	Not Started	An adversary car	Information Disc	An adversary car		Seller Request	Do not expose se	High	Implementation
45	Diagram 1		Generated	Not Started	An adversary ma	Information Disc	An adversary ma		Seller Request	Ensure that the a	High	Implementation
46	Diagram 1		Generated	Not Started	An adversary ma	Information Disc	An adversary ma		Seller Request	Ensure that sensi	High	Implementation
47	Diagram 1		Generated	Not Started	An adversary car	Information Disc	Robots.txt is ofte		Seller Request	Ensure that admi	Medium	Implementation
48	Diagram 1		Generated	Not Started	An adversary car	Information Disc	An adversary ma		Seller Request	Applications ava	High	Implementation
49	Diagram 1		Generated	Not Started	An adversary car	Information Disc	An adversary car		Seller Request	Do not expose se	High	Implementation
50	Diagram 1		Generated	Not Started	An adversary ma	Information Disc	An adversary ma		Seller Request	Ensure that sensi	High	Implementation

Threat List												
ID	Diagram	Changed By	Last Modified	State	Title	STRIDE Category	Description	Justification	Interaction	Possible Mitig	Severity	SDL Phase
51	Diagram 1		Generated	Not Started	Attacker can den	Repudiation	Proper logging o		Seller Request	Ensure that audit	Medium	Implementation
52	Diagram 1		Generated	Not Started	An adversary car	Spoofing	The session cook		Seller Request	Set up session fo	High	Implementation
53	Diagram 1		Generated	Not Started	An adversary car	Spoofing	The session cook		Seller Request	Enable ValidateR	High	Implementation
54	Diagram 1		Generated	Not Started	An adversary car	Spoofing	Ensure that TLS c		Seller Request	Verify X.509 certi	High	Implementation
55	Diagram 1		Generated	Not Started	An adversary car	Spoofing	Attackers can exj		Seller Request	Explicitly disable	High	Implementation
56	Diagram 1		Generated	Not Started	Attackers can ste	Spoofing	The session cook		Seller Request	Applications ava	High	Implementation
57	Diagram 1		Generated	Not Started	An adversary car	Spoofing	Phishing is atterr		Seller Request	Verify X.509 certi	High	Implementation
58	Diagram 1		Generated	Not Started	An adversary ma	Spoofing	If proper authent		Seller Request	Consider using a	High	Design
59	Diagram 1	DESKTOP-MOG8	10/6/2026 12:43:4	Mitigated	An adversary car	Tampering	Website defacer	Mitigated by listi	Seller Request	Implement Cont	High	Implementation
60	Diagram 1		Generated	Not Started	An attacker steal	Tampering	An attacker steal		Seller Request		High	Implementation
61	Diagram 1		Generated	Not Started	An adversary car	Tampering	SQL injection is z		Seller Request	Ensure that type-	High	Implementation
62	Diagram 1	DESKTOP-MOG8	10/6/2026 12:43:4	Mitigated	An adversary car	Tampering	An adversary car	Mitigated by listi	Seller Request	Encrypt sections	High	Implementation
63	Diagram 1		Generated	Not Started	An adversary car	Denial of Service	Failure to restrict		Admin Request	Ensure that auth	High	Implementation
64	Diagram 1	DESKTOP-MOG8	10/6/2026 12:43:4	Mitigated	An adversary ma	Elevation of Privi	Failure to restrict	Mitigated by adr	Admin Request	Ensure that admi	High	Implementation
65	Diagram 1		Generated	Not Started	An adversary car	Information Disc	An adversary car		Admin Request	Do not expose se	High	Implementation
66	Diagram 1		Generated	Not Started	An adversary ma	Information Disc	An adversary ma		Admin Request	Ensure that the a	High	Implementation
67	Diagram 1		Generated	Not Started	An adversary ma	Information Disc	An adversary ma		Admin Request	Ensure that sensi	High	Implementation
68	Diagram 1		Generated	Not Started	An adversary car	Information Disc	Robots.txt is ofte		Admin Request	Ensure that admi	Medium	Implementation
69	Diagram 1		Generated	Not Started	An adversary car	Information Disc	An adversary ma		Admin Request	Applications ava	High	Implementation
70	Diagram 1		Generated	Not Started	An adversary car	Information Disc	An adversary car		Admin Request	Do not expose se	High	Implementation
71	Diagram 1		Generated	Not Started	An adversary ma	Information Disc	An adversary ma		Admin Request	Ensure that sensi	High	Implementation
72	Diagram 1	DESKTOP-MOG8	10/6/2026 12:44:0	Mitigated	Attacker can den	Repudiation	Proper logging o	Mitigated by auc	Admin Request	Ensure that audit	Medium	Implementation
73	Diagram 1		Generated	Not Started	An adversary car	Spoofing	The session cook		Admin Request	Set up session fo	High	Implementation
74	Diagram 1		Generated	Not Started	An adversary car	Spoofing	The session cook		Admin Request	Enable ValidateR	High	Implementation
75	Diagram 1		Generated	Not Started	An adversary car	Spoofing	Ensure that TLS c		Admin Request	Verify X.509 certi	High	Implementation
76	Diagram 1		Generated	Not Started	An adversary car	Spoofing	Attackers can exj		Admin Request	Explicitly disable	High	Implementation
77	Diagram 1		Generated	Not Started	Attackers can ste	Spoofing	The session cook		Admin Request	Applications ava	High	Implementation
78	Diagram 1		Generated	Not Started	An adversary car	Spoofing	Phishing is atterr		Admin Request	Verify X.509 certi	High	Implementation
79	Diagram 1		Generated	Not Started	An adversary ma	Spoofing	If proper authent		Admin Request	Consider using a	High	Design
80	Diagram 1		Generated	Not Started	An adversary car	Tampering	Website defacer		Admin Request	Implement Cont	High	Implementation
81	Diagram 1		Generated	Not Started	An attacker steal	Tampering	An attacker steal		Admin Request		High	Implementation
82	Diagram 1		Generated	Not Started	An adversary car	Tampering	SQL injection is z		Admin Request	Ensure that type-	High	Implementation
83	Diagram 1		Generated	Not Started	An adversary car	Tampering	An adversary car		Admin Request	Encrypt sections	High	Implementation
84	Diagram 1		Generated	Not Started	An adversary car	Elevation of Privi	If there is no rest		Application Data	Configure a Win	High	Implementation
85	Diagram 1		Generated	Not Started	An adversary car	Elevation of Privi	Database access		Application Data	Ensure that least	High	Implementation
86	Diagram 1	DESKTOP-MOG8	10/6/2026 12:44:1	Mitigated	An adversary car	Information Disc	An adversary car	Mitigated by role	Application Data	Ensure SQL serve	High	Implementation
87	Diagram 1		Generated	Not Started	An adversary car	Information Disc	Additional contri		Application Data	Use strong encry	High	Implementation
88	Diagram 1	DESKTOP-MOG8	10/6/2026 12:44:1	Mitigated	An adversary car	Information Disc	SQL injection is z	Mitigated by role	Application Data	Ensure that login	High	Implementation
89	Diagram 1		Generated	Not Started	An adversary car	Repudiation	Proper logging o		Application Data	Ensure that login	Medium	Implementation
90	Diagram 1		Generated	Not Started	An adversary car	Tampering	An adversary car		Application Data	Add digital signa	High	Design
91	Diagram 1		Generated	Not Started	An adversary ma	Tampering	An adversary ma		Application Data	Enable Threat de	High	Design
92	Diagram 1		Generated	Not Started	An adversary car	Information Disc	An adversary car		Application Data	Do not expose se	High	Implementation
93	Diagram 1		Generated	Not Started	An adversary ma	Information Disc	An adversary ma		Application Data	Ensure that the a	High	Implementation
94	Diagram 1		Generated	Not Started	An adversary car	Information Disc	An adversary car		Application Data	Do not expose se	High	Implementation
95	Diagram 1		Generated	Not Started	Attacker can den	Repudiation	Proper logging o		Application Data	Ensure that audit	Medium	Implementation
96	Diagram 1		Generated	Not Started	An adversary car	Spoofing	Ensure that TLS c		Application Data	Verify X.509 certi	High	Implementation
97	Diagram 1		Generated	Not Started	An adversary car	Spoofing	Attackers can exj		Application Data	Explicitly disable	High	Implementation
98	Diagram 1		Generated	Not Started	An adversary car	Spoofing	Phishing is atterr		Application Data	Verify X.509 certi	High	Implementation
99	Diagram 1		Generated	Not Started	An adversary ma	Spoofing	If proper authent		Application Data	Consider using a	High	Design
100	Diagram 1	DESKTOP-MOG8	10/6/2026 12:44:2	Mitigated	An adversary car	Tampering	SQL injection is z	Mitigated by SQL	Application Data	Ensure that type-	High	Implementation

ID	Diagram	Changed By	Last Modified	State	Title	STRIDE Categ	Description	Justification	Interaction	Possible Mitig	Severity	SDL Phase
101	Diagram 1	DESKTOP-MOG8	10/6/2026 12:44:	Mitigated	An adversary car	Tampering	An adversary car	Mitigated by file	Application Data	Encrypt sections	High	Implementation
102	Diagram 1		Generated	Not Started	An adversary car	Elevation of Privi	An adversary car		Image Store/Retr	Grant limited acc	High	Implementation
103	Diagram 1		Generated	Not Started	An adversary car	Elevation of Privi	An adversary car		Image Store/Retr	It is recommend	High	Implementation
104	Diagram 1		Generated	Not Started	An adversary ma	Elevation of Privi	An adversary ma		Image Store/Retr	Assign the appro	High	Implementation
105	Diagram 1		Generated	Not Started	An adversary car	Elevation of Privi	An adversary car		Image Store/Retr	Ensure secure m	High	Implementation
106	Diagram 1		Generated	Not Started	An adversary car	Information Disc	An adversary car		Image Store/Retr	Ensure that com	Medium	Implementation
107	Diagram 1	DESKTOP-MOG8	10/6/2026 12:46:	Not Started	An adversary car	Repudiation	Proper logging o	Not applicable b	Image Store/Retr	Use Azure Stora	Medium	Implementation
108	Diagram 1		Generated	Not Started	An adversary car	Elevation of Privi	An adversary car		Image Store/Retr	Ensure that only	High	Implementation
109	Diagram 1		Generated	Not Started	An adversary car	Elevation of Privi	An adversary car		Image Result	Grant limited acc	High	Implementation
110	Diagram 1		Generated	Not Started	An adversary car	Elevation of Privi	An adversary car		Image Result	It is recommend	High	Implementation
111	Diagram 1		Generated	Not Started	An adversary ma	Elevation of Privi	An adversary ma		Image Result	Assign the appro	High	Implementation
112	Diagram 1		Generated	Not Started	An adversary car	Elevation of Privi	An adversary car		Image Result	Ensure secure m	High	Implementation
113	Diagram 1		Generated	Not Started	An adversary car	Information Disc	An adversary car		Image Result	Ensure that com	Medium	Implementation
114	Diagram 1	DESKTOP-MOG8	10/6/2026 12:45:	Not Started	An adversary car	Repudiation	Proper logging o	Not applicable b	Image Result	Use Azure Stora	Medium	Implementation
115	Diagram 1		Generated	Not Started	An adversary car	Elevation of Privi	An adversary car		Image Result	Ensure that only	High	Implementation
116	Diagram 1		Generated	Not Started	An adversary car	Elevation of Privi	If there is no rest		Audit Log Read/	Configure a Win	High	Implementation
117	Diagram 1		Generated	Not Started	An adversary car	Elevation of Privi	Database access		Audit Log Read/	Ensure that least	High	Implementation
118	Diagram 1		Generated	Not Started	An adversary car	Information Disc	An adversary car		Audit Log Read/	Ensure SQL serve	High	Implementation
119	Diagram 1		Generated	Not Started	An adversary car	Information Disc	Additional contn		Audit Log Read/	Use strong encry	High	Implementation
120	Diagram 1		Generated	Not Started	An adversary car	Information Disc	SQL injection is z		Audit Log Read/	Ensure that login	High	Implementation
121	Diagram 1	DESKTOP-MOG8	10/6/2026 12:44:	Mitigated	An adversary car	Repudiation	Proper logging o	Mitigated by ma	Audit Log Read/	Ensure that login	Medium	Implementation
122	Diagram 1	DESKTOP-MOG8	10/6/2026 12:44:	Mitigated	An adversary car	Tampering	An adversary car	Mitigated by rest	Audit Log Read/	Add digital signa	High	Design
123	Diagram 1	DESKTOP-MOG8	10/6/2026 12:44:	Mitigated	An adversary ma	Tampering	An adversary ma	Mitigated by rest	Audit Log Read/	Enable Threat de	High	Design
124	Diagram 1		Generated	Not Started	An adversary car	Information Disc	An adversary car		Audit Log Result	Do not expose se	High	Implementation
125	Diagram 1		Generated	Not Started	An adversary ma	Information Disc	An adversary ma		Audit Log Result	Ensure that the a	High	Implementation
126	Diagram 1		Generated	Not Started	An adversary car	Information Disc	An adversary car		Audit Log Result	Do not expose se	High	Implementation
127	Diagram 1	DESKTOP-MOG8	10/6/2026 12:44:	Mitigated	Attacker can den	Repudiation	Proper logging o	Mitigated by ma	Audit Log Result	Ensure that audit	Medium	Implementation
128	Diagram 1		Generated	Not Started	An adversary car	Spoofing	Ensure that TLS c		Audit Log Result	Verify X.509 cert	High	Implementation
129	Diagram 1		Generated	Not Started	An adversary car	Spoofing	Attackers can exj		Audit Log Result	Explicitly disable	High	Implementation
130	Diagram 1		Generated	Not Started	An adversary car	Spoofing	Phishing is atten		Audit Log Result	Verify X.509 cert	High	Implementation
131	Diagram 1		Generated	Not Started	An adversary ma	Spoofing	If proper authent		Audit Log Result	Consider using a	High	Design
132	Diagram 1		Generated	Not Started	An adversary car	Tampering	SQL injection is z		Audit Log Result	Ensure that type	High	Implementation
133	Diagram 1		Generated	Not Started	An adversary car	Tampering	An adversary car		Audit Log Result	Encrypt sections	High	Implementation

Figure E-2.1: STRIDE Categories Table Generated STRIDE Threat List from Microsoft Threat Modeling Tool

Threat Properties		Status: Mitigated	Last Modified: DESKTOP-MOG84MD\admin 10/6/2026 12:42:13 am
ID: 0	Diagram: Diagram 1		
Title:	An adversary can perform action on behalf of other user due to lack of controls against cross domain requests		
STRIDE Category:	Denial of Service		
Description:	Failure to restrict requests originating from third party domains may result in unauthorized actions or access of data		
Justification:	Mitigated through planned rate limiting, request throttling, pagination limits, bot detection, and excessive request monitoring on login, search, upload, checkout, and dispute endpoints.		
Interaction:	Guest Request		
Possible Mitigation(s):	Ensure that authenticated ASP.NET pages incorporate UI Redressing or clickjacking defences. Refer: https://aka.ms/tmtconfigmgmt#ui-defenses Ensure that only trusted origins are allowed if CORS is enabled on ASP.NET Web Applications. Refer: https://aka.ms/tmtconfigmgmt#cors-aspnet Mitigate against Cross-Site Request Forgery (CSRF) attacks on ASP.NET web pages. Refer: https://aka.ms/tmtconfigmgmt#csrf-asp		
Severity:	High		
SDL Phase:	Implementation		

Figure E-2.2: Example of Threat Properties with Mitigation Note Added

Appendix F

Attack Surface Type	Entry / Exit Points
User Interface Forms	Registration Form, Login Form, Seller Application Form, Product Listing Form, Review Form, Dispute Form
Authentication Interfaces	Login Endpoint, Logout Endpoint, Session Cookies
Search Functions	Product Search, Search Filters
File Upload Interfaces	Product Image Upload
Transaction Workflows	Cart Management, Purchase Commitment, Simulated Checkout
Workflow Management	Shipment Status Updates, Authentication Status Updates
Administrative Interfaces	Admin Dashboard, User Management, Listing Moderation, Dispute Management
Data Stores	User Database, Product Database, Order Database, Review Database, Audit Log Database
File Storage	Uploaded Product Images
System Logs	Audit Logging System

Table F1: Attack Surface Identification

Appendix G

Appendix G-1 - High-Level Architecture

Layer	Components	Purpose
Client Layer	Guest browser, buyer browser, seller browser, administrator browser	Allows users to access the marketplace through a web browser
Edge Protection Layer	HTTPS/TLS, reverse proxy, security headers, rate limiting	Protects transport, reduces exposed attack surface and filters abnormal requests
Web Application Layer	Flask routes, Jinja2 templates, sessions, CSRF protection, authentication and authorisation	Handles user requests and enforces application-level security
Business Service Layer	User, seller, listing, upload, cart, order, workflow, review, dispute, admin and audit services	Enforces business rules and security decisions
Data and Storage Layer	SQLite database, uploaded image storage, audit log storage and backup storage	Stores protected system data and security evidence

Table G-1.1: Five Layers of High-Level Architecture

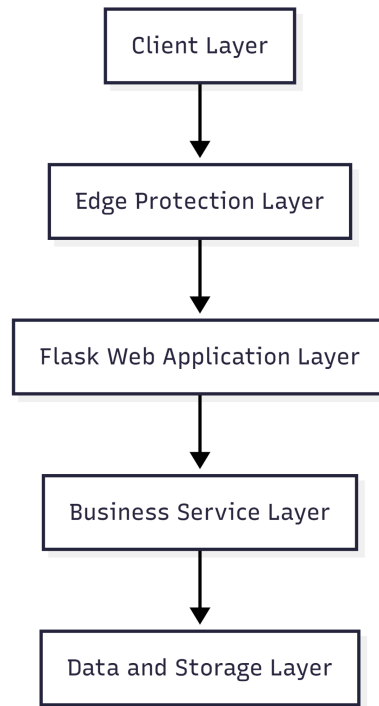


Figure G-1.1: High-Level Architecture Diagram

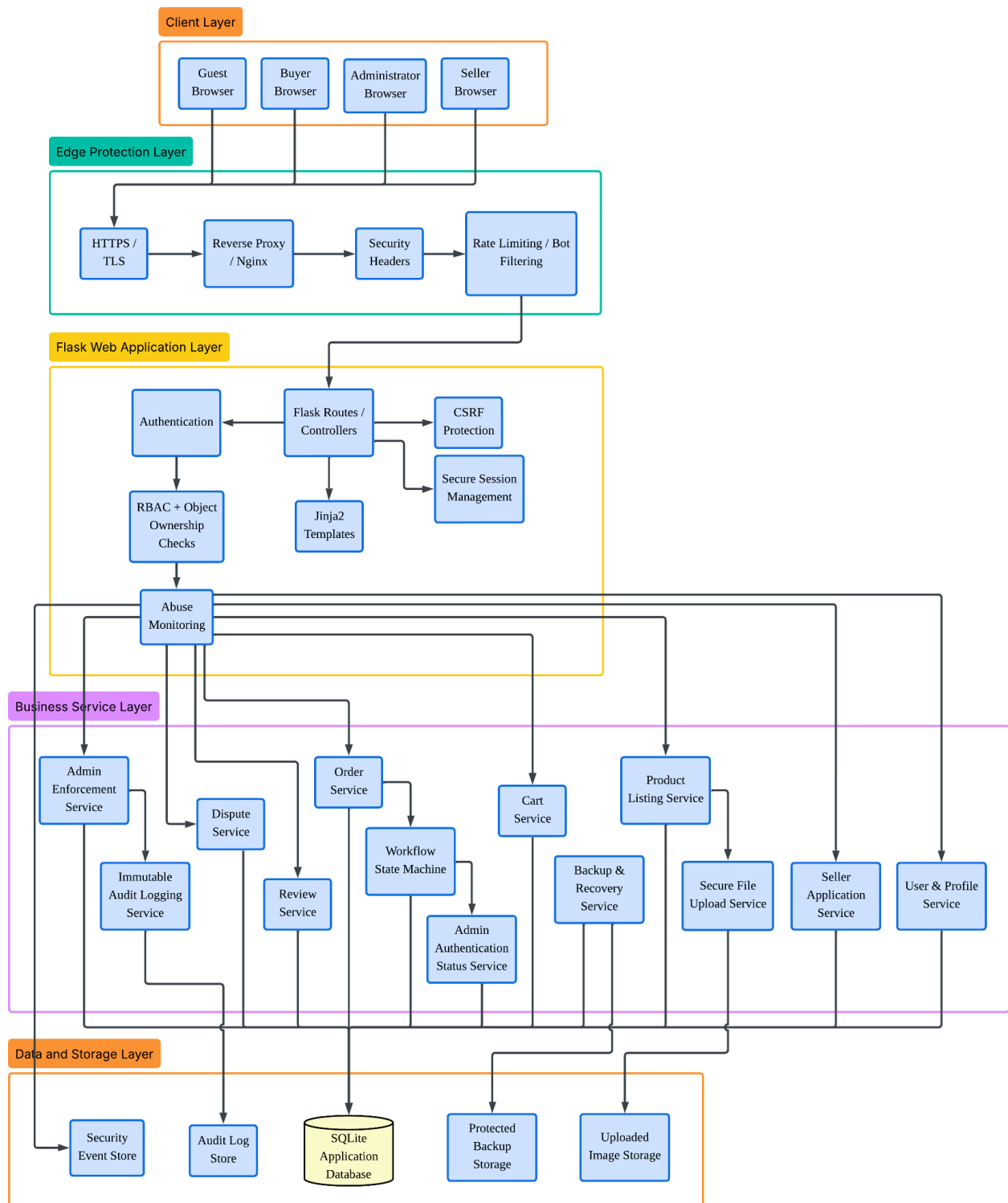


Figure G-1.2: High-Level Architecture Diagram with Components

Appendix G-2 - Logical Architecture

Trust Zone	Components	Security Concerns
External Users / Untrusted Requests	Guest Browser, Buyer Browser, Seller Browser, Administrator Browser, Bot / Attacker	Untrusted input, fake registration, credential stuffing, account takeover, scraping, request tampering, user enumeration and bot traffic.
Flask Web Application	HTTPS, reverse proxy, security headers, rate limiting, Flask routes, Jinja2 templates, session management, CSRF protection, authentication, RBAC and object ownership checks	Broken authentication, session abuse, CSRF, injection, XSS, IDOR, insecure direct access, excessive requests and security misconfiguration.
Backend Business Logic and Workflow Processing	User and Profile Service, Seller Application Service, Product Listing Service, Secure File Upload Service, Cart Service, Order and Workflow Service, Admin Authentication Status Service, Review Service and Dispute Service	Seller fraud, counterfeit listing abuse, malicious uploads, price tampering, workflow bypass, false shipment updates, fake reviews, invalid disputes and business-rule bypass.
Privileged Administrative Functions	Admin Enforcement Service, User Suspension, Listing Moderation, Workflow and Authentication Status Updates, Dispute Resolution and Audit Log Viewer	Unauthorised admin access, privilege escalation, admin account takeover, insider abuse, unauthorised workflow changes and audit log tampering.
Protected Backend Storage	SQLite Application Database, Uploaded Image Storage, Audit Log Store, Security Event Store and Protected Backup Storage	Unauthorised database access, information disclosure, data tampering, unsafe file access, audit log manipulation, backup exposure and data loss.

Table G-2.1: Logical Trust Zones

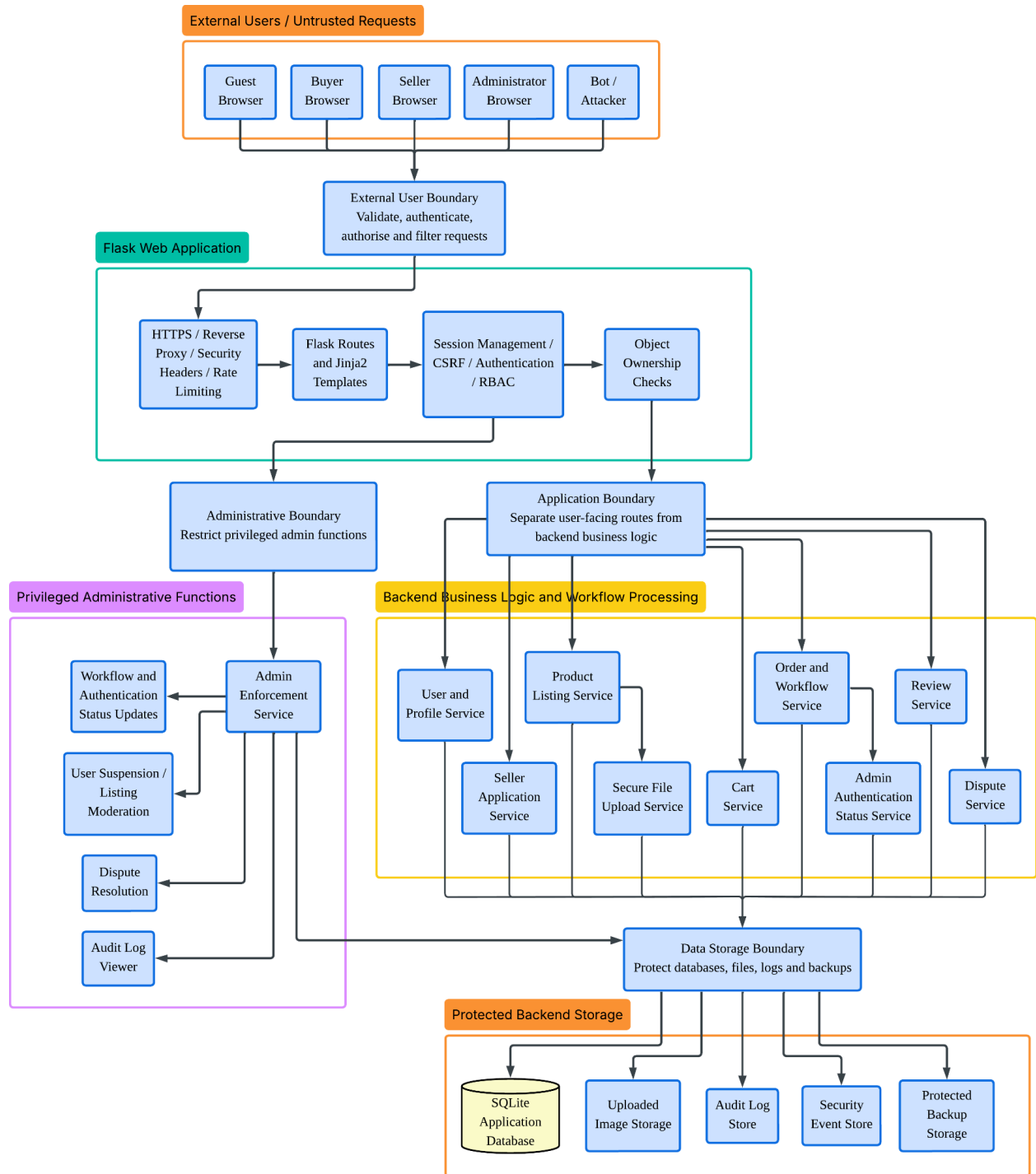


Figure G-2.1: Logical Architecture Diagram

Trust Boundary	Boundary Crossing	Required Controls	Main Threats Addressed
External User Boundary	External browsers and bots send requests to the Flask web application.	HTTPS/TLS, reverse proxy controls, security headers, rate limiting, bot filtering, input validation, authentication, authorisation, CSRF protection, secure cookies and session expiry.	Account takeover, credential stuffing, fake registration, scraping, request flooding, CSRF, injection, XSS and unauthorised access.
Application Boundary	User-facing Flask routes and templates call backend business services.	Centralised business services, server-side validation, RBAC, ownership checks, parameter tampering protection, state-transition validation and rejection of client-controlled price, role, order status and authentication status values.	IDOR, price tampering, workflow bypass, mass assignment, unauthorised record access and business-rule bypass.
Administrative Boundary	Authenticated users attempt to access privileged administrative functions.	Administrator authentication, 2FA, RBAC, least privilege, sensitive-action confirmation, activity monitoring, audit logging and restricted audit log access.	Privilege escalation, unauthorised admin access, admin account takeover, insider abuse, false workflow updates and audit log tampering.
Data Storage Boundary	Application and admin services access SQLite, uploaded files, logs and backups.	ORM or parameterised queries, least-privilege file permissions,	Unauthorised database access, information disclosure, SQL injection, data

Trust Boundary	Boundary Crossing	Required Controls	Main Threats Addressed
		protected storage paths, non-public upload directories, server-generated filenames, file validation, audit log protection, backup protection and recovery controls.	tampering, unsafe file retrieval, audit log manipulation and backup exposure.

Table G-2.2: Trust-Boundary Control Mapping

Appendix G-3 - Physical Architecture

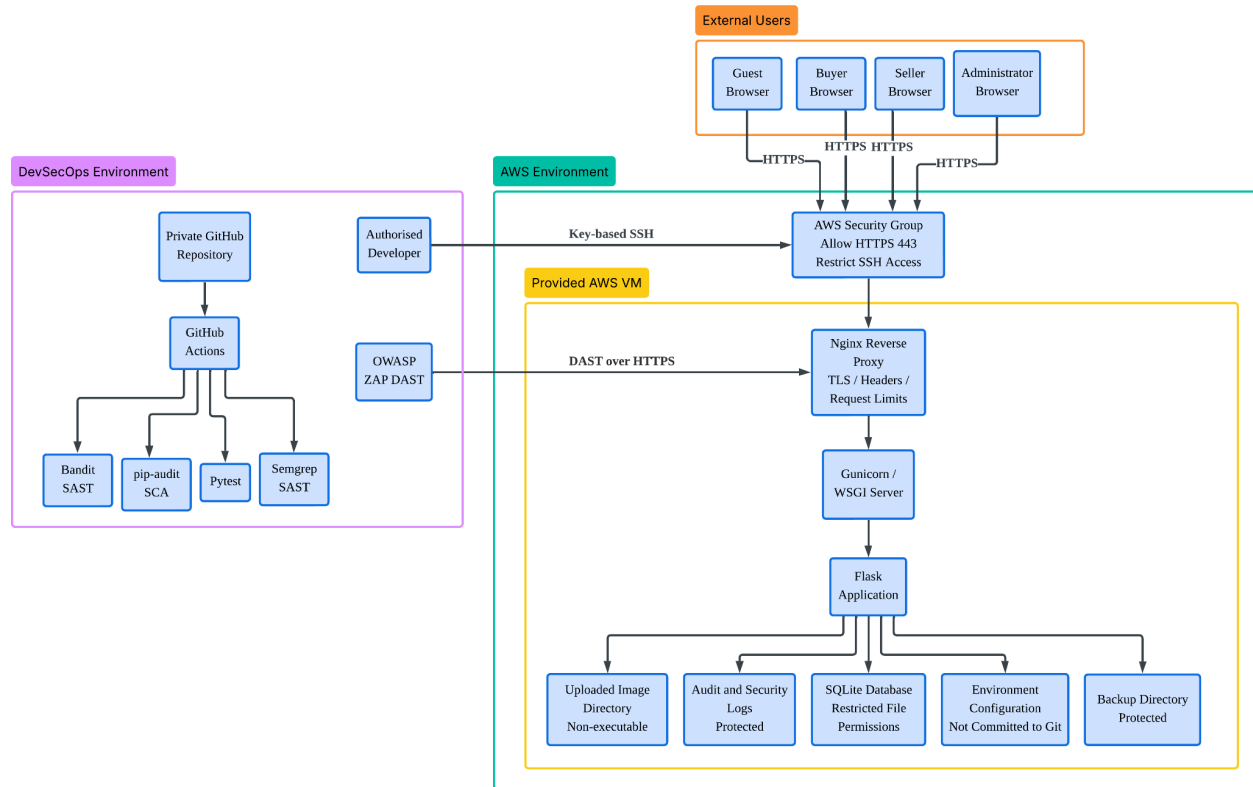


Figure G-3.1: Physical Architecture Diagram

Storage Area	Example Location	Protection
Application code	/var/www/chateau-collective	Restricted to application/deployment user
SQLite database	/var/lib/chateau-collective/db	Not publicly accessible; restricted file permissions
Uploaded images	/var/lib/chateau-collective/uploads	Outside executable web root; server-generated filenames
Audit logs	/var/log/chateau-collective/audit	Append-only design where feasible; admin/system access only

Storage Area	Example Location	Protection
Security event logs	/var/log/chateau-collective/security	Restricted access; used for suspicious activity review
Backups	/var/backups/chateau-collective	Not publicly accessible; protected server permissions
Environment configuration	.env or server environment variables	Not committed to GitHub; restricted access

Table G-3.1: Physical Storage Layout

Appendix G-4 - Component Architecture

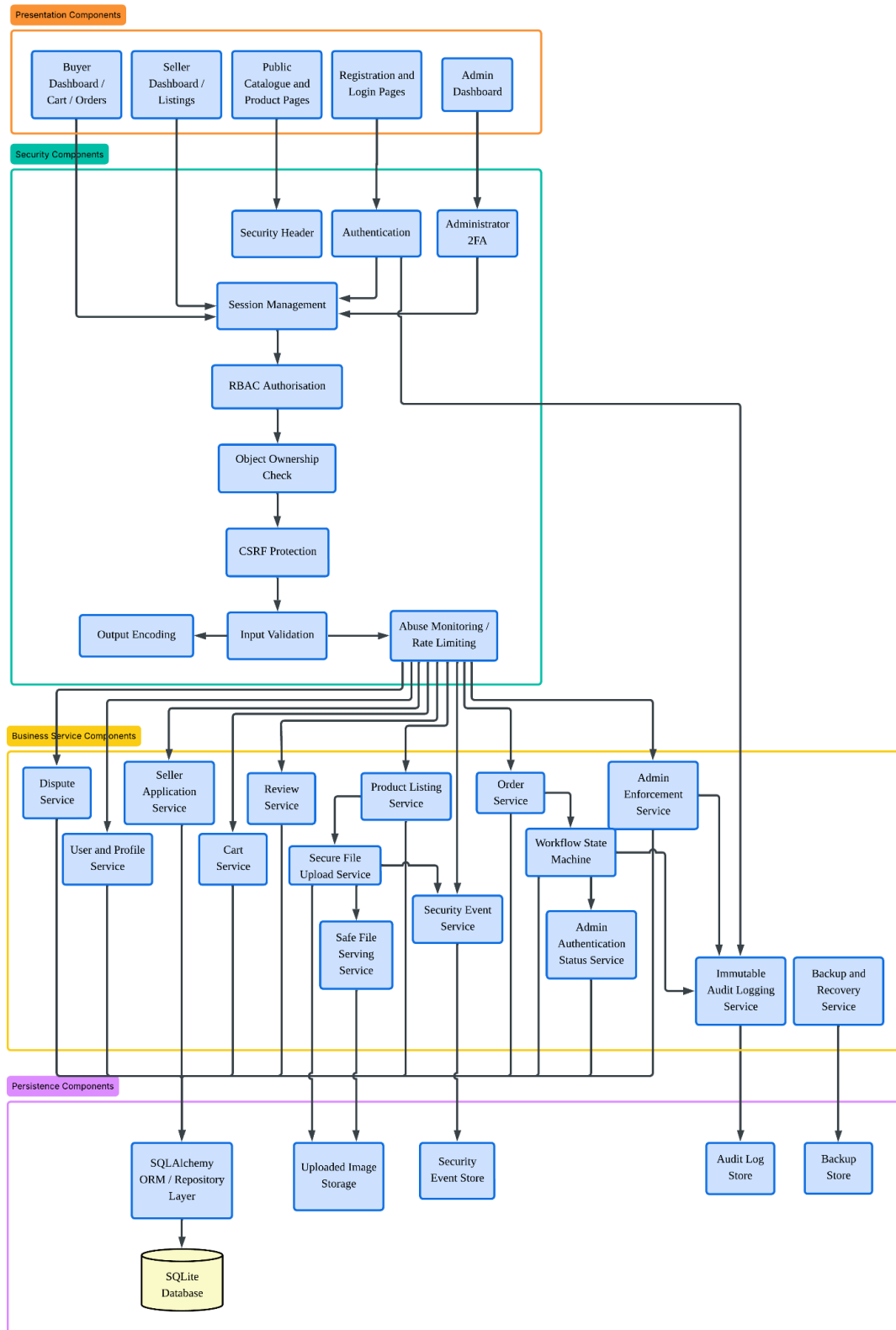


Figure G-4.1: Component Diagram

Component	Responsibility	Main Security Contribution
Authentication Component	Handles login, logout and password verification	Prevents unauthenticated access
Administrator Component 2FA	Requires additional verification for administrator access	Reduces admin account takeover risk
Session Management Component	Creates, validates and expires authenticated sessions	Prevents session misuse and stale login sessions
RBAC Authorisation Component	Enforces Guest, Buyer, Seller and Administrator permissions	Prevents role-based privilege escalation
Object Ownership Check Component	Verifies that users only access records they own or are authorised to manage	Prevents IDOR across profiles, carts, orders, reviews and disputes
CSRF Protection Component	Protects state-changing requests	Prevents forged authenticated actions
Input Validation Component	Validates forms, search filters, workflow requests and uploads	Reduces injection, tampering and invalid data risks
Output Encoding Component	Encodes user-generated content before display	Reduces stored and reflected XSS risks
Abuse Monitoring Component	Detects excessive registration, login, search, upload, checkout and dispute activity	Reduces brute force, scraping, bot and DoS risks
Secure File Upload Service	Validates uploaded image files before storage	Reduces malicious file upload and path traversal risks
Safe File Serving Service	Serves uploaded images safely without allowing execution	Prevents unsafe public exposure of uploaded content

Component	Responsibility	Main Security Contribution
Workflow State Machine	Enforces valid order, shipment, authentication and checkout transitions	Prevents workflow skipping and status manipulation
Admin Authentication Status Service	Allows only administrators to update simulated luxury goods authentication status	Prevents sellers from self-authenticating items
Immutable Audit Logging Service	Records important user, seller, buyer and admin actions	Supports accountability and investigation
Security Event Service	Records suspicious security activity	Supports monitoring and incident review
Backup and Recovery Service	Supports recovery of important system data	Supports availability and resilience

Table G-4.1: Component Responsibilities

Appendix H

Appendix H-1 - Security Design Principles

Principle	How It Is Enforced
Least Privilege	Each role (Guest, Buyer, Seller) has the minimal permissions required. Admin is separated for role management.
Defense in Depth	Validation at UI, controller, service, and DB layers; file validation + renaming + safe storage; session management + token + role checks; security headers enforced on all responses (FSR-19).
Deny by Default	Every protected endpoint requires explicit role, ownership, and workflow state checks. Safeguards are in place against broken access control.
Secure by Default	Simulated payment, server-side price recalculation, no client-trusted status fields.
Separation of Duties	Seller cannot authenticate items; Admin cannot bypass audit logs; Buyer cannot change workflow states.
Fail Securely	All errors return generic messages with no stack traces. On session expiry, the user is redirected to login. When validation fails, the operation is rejected.

Table H-1.1: Security Design Principles

Appendix H-2 - STRIDE Categories

STRIDE Category	Threat Scenario	Related Risk / Misuse Case	Mitigation
Spoofing	An attacker uses stolen credentials to access buyer, seller, or admin accounts.	Account and Session Abuse; Brute Force / Credential Stuffing	Enforce strong password policy, secure password hashing, login throttling, and secure session management.
Spoofing	An attacker creates fake accounts or uses credential stuffing to take over buyer, seller, or admin accounts.	Account Takeover, Fake Registration and Profile IDOR	Apply registration and login rate limiting, strong password policy, secure password hashing, login throttling, secure session management, and suspicious activity logging.
Tampering	A buyer modifies price, order status, checkout value, or workflow state through request manipulation.	Cart and Checkout Bypass; Cart and Checkout Tampering	Validate prices, ownership, and workflow states on the server side. Client-side values for price, role, order status, and authentication status must not be trusted.
Tampering	A seller uploads a malicious file disguised as a product image.	Fraudulent Seller and Listing Abuse; Malicious File Upload	Validate file type, extension, MIME type, and file size. Rename uploaded files using server-generated filenames and store them safely.
Tampering	A seller attempts to mark their own item as authenticated or bypass the admin-managed workflow.	False Status Updates; Workflow State Manipulation	Only administrators can update simulated authentication status. Valid workflow state transitions must be enforced server-side.
Tampering	A user changes object IDs in requests to access or modify another user's profile, cart, order, listing, shipment, review, or dispute record.	Profile IDOR; Order Ownership Bypass; Invalid Dispute Abuse	Enforce server-side object ownership checks and deny-by-default access control for private records.

Tampering	A seller bypasses seller approval or listing approval to publish a counterfeit or misleading listing.	Fraudulent Seller, Counterfeit Listing and Malicious Upload Abuse	Require admin review, enforce seller role checks, enforce listing ownership checks, and keep second-hand listings hidden until approval.
Tampering	A buyer submits a review without a completed purchase, raises a dispute for another user's order, or tampers with dispute details.	Fake Review, Invalid Dispute and Dispute Abuse	Verify purchase completion, enforce order ownership checks, restrict duplicate reviews, require dispute details, and log review/dispute actions.
Tampering	A buyer forces checkout or order completion without satisfying required workflow states.	Price Tampering, Order Ownership Bypass and Workflow Skipping	Enforce server-side workflow validation and reject invalid state transitions.
Tampering / Repudiation	An attacker, compromised admin, or malicious admin modifies or deletes audit logs to hide misuse.	Admin Account Takeover, Abusive Admin Action and Audit Log Tampering	Restrict audit log access, protect logs from unauthorised modification or deletion, and record admin identity, timestamp, action type, and target record.
Repudiation	A seller or admin denies performing a listing, shipment, workflow, authentication status, or dispute action.	Admin Abuse; False Status Updates	Record audit logs with actor ID, timestamp, action type, old/new state, and target record.
Information Disclosure	A user accesses another user's profile, cart, order, dispute, or private records.	Sensitive Data / PII Exposure	Apply role-based access control, ownership checks, and deny-by-default access rules.
Information Disclosure	An attacker uses search injection, listing enumeration, or scraping to discover hidden listings,	Search Injection, Listing Enumeration and Scraping Abuse	Validate search inputs, use parameterised queries, return only approved public listings,

	unapproved listings, internal listing flags, admin notes, or private seller information.		hide internal metadata, and apply scraping controls.
Denial of Service	A bot floods login, search, upload, checkout, or dispute endpoints.	Catalog Scraping; Search / API Flooding	Apply rate limiting, request throttling, pagination limits, bot detection, and excessive request monitoring.
Elevation of Privilege	A buyer or seller accesses admin functions or modifies role values in requests.	Unauthorized Admin Access / Privilege Escalation	Protect admin routes with RBAC middleware. Verify roles server-side and never trust client-submitted role values.
Elevation of Privilege	A regular user attempts to perform seller-only actions before seller approval.	Fraudulent Seller and Counterfeit Listing Abuse	Enforce seller verification and role checks before listing creation or editing.
Elevation of Privilege	A compromised or malicious admin performs actions beyond intended administrative authority.	Admin Account Takeover, Abusive Admin Action and Audit Log Tampering	Apply least privilege, RBAC, MFA, sensitive action confirmation, and unusual admin activity monitoring.

Table H-2.1: STRIDE Categories

Appendix H-3 - Workflow State Design

State	Description	Authorised Actor
Available	Approved product is visible and available for purchase	System / Admin
Committed	Buyer has committed to purchase the item	Buyer
Awaiting Shipment	Seller is expected to ship the item after buyer commitment	System / Seller
Shipped	Seller has marked the item as shipped	Seller
Under Authentication	Item is being reviewed through the simulated in-house authentication workflow	Admin
Authenticated	Item passes simulated authentication	Admin
Rejected	Item fails simulated authentication or workflow compliance	Admin
Sold	Checkout is completed after required workflow checks are satisfied	System / Buyer

Table H-3.1: Workflow States

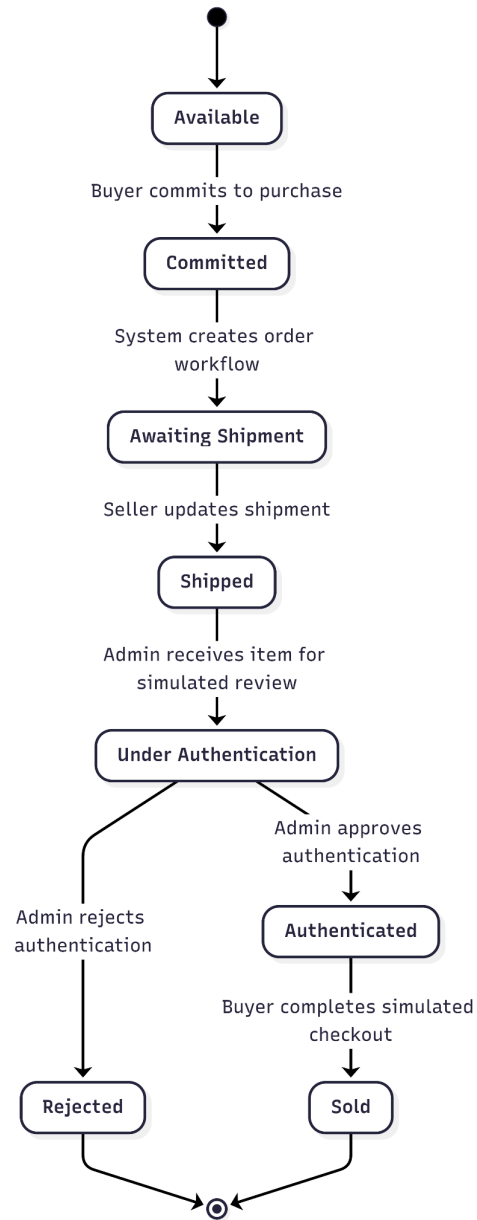


Figure H-3.1: Workflow State Diagram

Appendix H-4 - Class Diagram

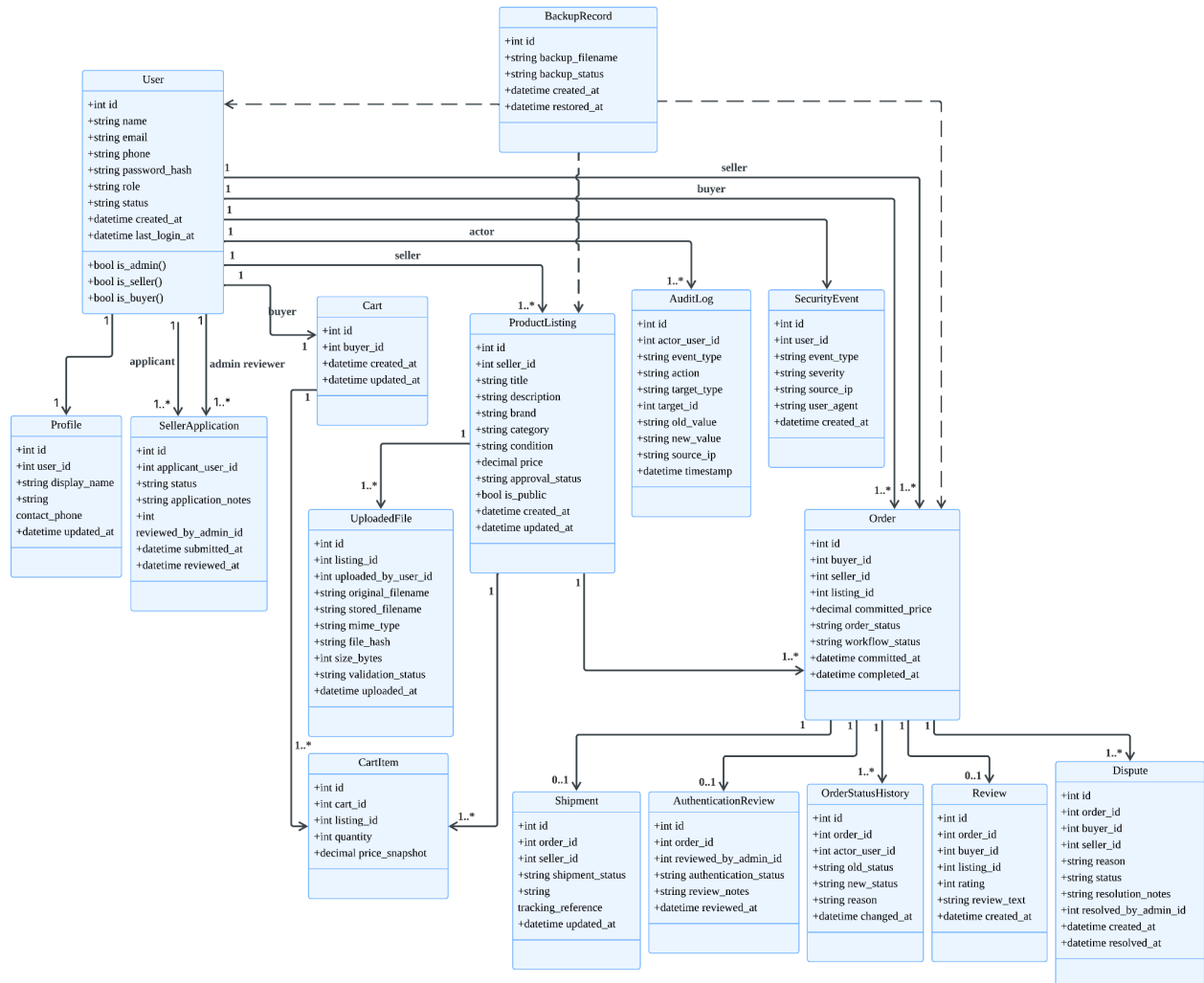


Figure H-4.1: System Class Diagram